

國立臺北商業大學

資訊管理系

114 資訊系統專案設計

系統手冊



組別：第 114411 組

題目：SignNest-手語小窩

指導老師：林俊杰老師

組長：11046042 周駿翔

組員：11146003 黃俊翰 11146038 郭子寧

11146040 黃政

中華民國 114 年 12 月 31 日

國立臺北商業大學專題課程作品

電子文件上網授權書

本授權書所授權之作品為授權人在 國立臺北商業大學 資訊管理系

111411 組 114 學年度 第 2 學期專題課程之成果報告作品。

專題題目：SignNest-手語小窩

指導教授：林俊杰

☐ 本人 ☐ 團體著作人

☐ 同意 提供讀者基於個人非營利性質之線上檢索、閱覽、下載或列印。

茲同意將授權人擁有著作權之上列論文全文(含摘要)無償授權本人就讀學校(國立臺北商業大學)圖書館，不限地域、時間與次數，以微縮、光碟或其他各種數位化方式將上列作品收錄、重製與利用，並得將數位化之上列作品及其電子檔上載資料庫系統。於著作權法合理使用範圍內，讀者得進行線上檢索、閱覽、下載或列印。

專題課程成果報告作品全文上載網路公開之範圍及時間：

本校區域網路 ☐ 立即公開

☐ 自中華民國 年 月 日公開

校外網際網路 ☐ 立即公開

☐ 自中華民國 年 月 日公開

☐ 不同意 提供讀者基於個人非營利性質之線上檢索、閱覽、下載或列印。

授權人

簽 名

中 華 民 國 1 1 4 年 1 2 月 3 1 日

(作品若為團體作品，請每一位團體成員均應簽署)

目錄

第一章 前言

1-1 背景介紹.....	8
1-2 動機.....	8
1-3 系統目的與目標.....	9
1-4 預期成果.....	9

第二章 營運計畫

2-1 可行性分析.....	10
2-2 商業模式－Business model.....	13
2-3 市場分析－STP.....	14
2-4 競爭力分析 SWOT-TOWS	15

第三章 系統規格

3-1 系統架構.....	17
3-2 系統軟、硬體需求與技術平台.....	24
3-3 使用標準與工具.....	24

第四章 專案時程與組織分工

4-1 專案時程.....	25
4-2 專案組織與分工.....	26
4-3 上傳 GitHub 紀錄.....	29

第五章 需求模型

5-1 使用者需求.....	30
----------------	----

5-2 使用個案圖(Use case diagram).....	31
5-3 使用個案描述.....	32
5-4 分析類別圖(Analysis class diagram).....	34

第六章 設計模型

6-1 循序圖(Sequential diagram).....	35
6-2 設計類別圖(Design class diagram).....	37

第七章 實作模型

7-1 佈署圖(Deployment diagram).....	38
7-2 套件圖(Package diagram).....	38
7-3 元件圖(Component diagram).....	39
7-4 狀態機(State machine).....	39

第八章 資料庫設計

8-1 資料庫關聯表.....	40
8-2 表格及其 Meta data.....	40

第九章 程式

9-1 元件清單及其規格描述.....	42
9-2 其他附屬之各種元件.....	44

第十章 測試模型

10-1 測試計畫：說明採用之測試方法及其進行方式。.....	68
10-2 測試個案與測試結果資料。.....	69

第十一章 操作手冊

11-1 系統元件.....	73
11-2 系統需求.....	73
第十二章 使用手冊.....	74
第十三章 感想.....	79
第十四章 參考資料.....	82
附錄	
審查評審意見之修正情形.....	83

圖目錄

圖 3-1-1 系統架構.....	17
圖 3-1-2 可視化節點辨識紀錄.....	19
圖 3-1-3 節點 CSV 檔範例.....	19
圖 3-1-4 影片標註.....	20
圖 3-1-5 JSON 檔格式.....	21
圖 3-1-6 訓練結果.....	23
圖 3-1-7 訓練流程概念圖.....	23
圖 4-3-1 總體上傳.....	29
圖 4-3-2 個別上傳.....	29
圖 5-2-1 使用個案圖.....	31
圖 5-3-1 搜尋之活動圖.....	32
圖 5-3-2 課程觀看之活動圖.....	32
圖 5-3-3 管理課程之活動圖.....	33
圖 5-3-4 即時辨識之活動圖.....	34
圖 5-3-5 登入之活動圖.....	34
圖 5-4-1 分析類別圖(Analysis class diagram).....	35
圖 6-1-1 查看課程之循序圖.....	36
圖 6-1-2 搜尋之循序圖.....	37
圖 6-1-3 開啟即時辨識之循序圖.....	37
圖 6-2-1 設計類別圖(Design class diagram).....	38

圖 7-1-1 佈署圖(Deployment diagram).....	39
圖 7-2-1 套件圖(Package diagram).....	39
圖 7-3-1 元件圖(Component diagram).....	40
圖 7-4-1 狀態機(State machine).....	40
圖 8-1-1 資料庫關聯表.....	41

表目錄

表 2-1-1 時間可行性 (Schedule Feasibility)	12
表 2-2-1 商業模式—Business model.....	13
表 2-3-1 STP 分析.....	14
表 2-4-1 競爭力分析 SWOT-TOWS.....	16
表 3-3-1 使用標準與工具.....	24
表 4-1-1 專案時程.....	25
表 4-2-1 專案組織與分工.....	26
表 4-2-2 工作內容與貢獻度.....	28
表 5-1-1 功能性需求.....	30
表 5-1-2 非功能性需求.....	30
表 8-2-1 users 之表格.....	41
表 8-2-2 courses 之表格.....	42
表 8-2-3 lessons 之表格.....	42
表 8-2-4 gesture_word 之表格.....	42
表 9-1-1 前端程式清單.....	43
表 9-1-2 後端程式清單.....	43
表 9-1-2 其他元件檔案.....	43
表 9-2-1 extract_keypoints_3d.py.....	45
表 9-2-2 prepare_data_3d.py.....	49
表 9-2-3 train_transformer_3d.py.....	54

表 9-2-4 r_recognition_3d.py.....	59
表 10-2-1 會員注冊功能測試.....	70
表 10-2-2 會員登入功能測試.....	70
表 10-2-3 密碼重設功能測試.....	71
表 10-2-4 影片播放功能測試.....	71
表 10-2-5 詞彙分類功能測試.....	72
表 10-2-6 上傳教學影片功能測試.....	72
表 10-2-7 手語辨識功能測試.....	73
表 10-2-8 查詢詞語功能測試.....	73
表 11-1-1 系統安裝元件資訊.....	74
表 12-1-1 登入介面.....	75
表 12-1-2 註冊介面.....	75
表 12-1-3 首頁.....	76
表 12-1-4 關於我們介面.....	76
表 12-1-5 學習手語介面.....	77
表 12-1-6 即時翻譯手語介面.....	77
表 12-1-7 即時翻譯範例.....	78
表 12-1-8 使用者資訊介面.....	79

第 1 章 前言

1-1 背景介紹

由於本專題小組成員之一為聽障學生，我們在與他日常互動與課堂合作的過程中，深刻體會到溝通上的困難。由於我們不懂手語，經常需要他透過打字來傳達想法，不僅耗時，也容易因手語與中文語序不同而產生語句不順的情況，進一步影響理解與交流的流暢度。為了讓彼此的溝通更加即時且自然，我們希望能開發一套系統，讓聽障者能直接以手語表達，而系統能即時翻譯成文字，協助我們更清楚地理解其想法。相較於現今仍需依賴專業翻譯人員的溝通方式，這樣的系統不僅提升了自主性與便利性，也有助於推廣更平等的交流環境。

因此，我們決定以「即時視訊手語翻譯兼手語教學平台」為畢業專題題目，期望透過網頁系統結合電腦視覺與機器學習，實現手語即時辨識功能，並進一步推廣手語教學，改善聽障者與一般人之間的溝通障礙，提升整體資訊平權。

1-2 動機

我們的主要動機源於團隊成員中聽障學生的親身經歷與困境，讓我們深刻理解到手語翻譯資源的不足對聽障者造成的學習與生活挑戰。另一方面，我們觀察到目前尚無真正結合即時攝影機手語偵測與教學功能的整合式系統，這在技術上具有挑戰性，也極具發展潛力與社會價值。

此外，單一即時翻譯功能雖可解決基本溝通需求，但為了讓整體系統更具教育意義與可擴展性，我們增加了手語教學模組，以影片撥放的方式，讓使用者能透過搜尋功能找尋想學習的單詞來自我學習，並以日常生活及大學生常用詞彙來設計教材，以提升學習的實用性與情境連結，進而推廣手語文化，促進社會包容。

1-3 系統目的與目標

本專題旨在開發一個結合「即時手語翻譯」與「手語教學」功能的網頁平台，透過攝影機捕捉使用者的手勢影像，並運用人工智慧模型進行辨識與翻譯，讓使用者能即時看到所比手語對應的單詞內容。整體目標如下：

- 建立即時攝影機手勢辨識系統，翻譯使用者所比手語
- 發展手語教學模組，讓使用者能夠學習手語
- 提升非聽障者對手語的理解與學習意願，促進無障礙溝通環境

1-4 預期成果

本專題預計能在技術、教育與社會三個層面帶來具體成果。技術上，我們會完成一套結合攝影機與 AI 的即時手語辨識系統，並開發出具備手語教學功能的網頁平台。教育方面，這個平台可以協助一般使用者在沒有翻譯員時也能即時理解手語內容，同時讓一般使用者也能透過分級的學習方式輕鬆學會手語。社會層面來看，我們希望這個系統能提升大家對聽障族群的理解與包容，推動資訊平權，也作為未來在校園或社區中推廣手語學習的基礎工具。

第二章 營運計畫

2-1 可行性分析

2.1 技術可行性 (Technical Feasibility)

- 手語辨識技術：
 - 本系統採用 Google 開源的 MediaPipe Hands 偵測手部關鍵點，結合 MediaPipe Face Mesh 進行人臉辨識，並透過 OpenCV 進行影像前處理，TensorFlow 來訓練並調整模型參數，以提升整體辨識效果。

網站開發技術：

- 前端：HTML + CSS
- 後端：Python，Flask
- 資料存儲：資料直接寫入 MySQL，後端架構精簡

結論：技術上可行，可透過現有開源技術完成開發，符合需求。

2.2 經濟可行性 (Economic Feasibility)

- 開發工具成本：
 - 所有開發工具皆為免費開源 (MediaPipe、OpenCV、TensorFlow)。
- 設備需求：
 - 使用者需具備內建攝影機的電腦或手機，但這是大多數人已擁有的設備，因此不會增加使用門檻。

結論：經濟上可行，開發成本低，無需額外硬體或伺服器支出。

2.3 法律可行性（Legal Feasibility）

- 版權問題：
 - 網站上的教學內容將使用組員自己所拍攝的手語影片，避免侵犯智慧財產權。

結論：法律上可行，符合隱私與版權規範。

2.4 運營可行性（Operational Feasibility）

- 使用者體驗：
 - 即時手語辨識讓使用者無需下載額外應用程式，即可直接在瀏覽器中使用。
- 開發團隊技術能力：
 - 團隊具備前端開發與機器學習基礎，能夠學習並應用 MediaPipe、OpenCV、TensorFlow。

結論：運營上可行，技術門檻適中，適合大學專題開發。

2.5 時間可行性 (Schedule Feasibility)

階段	工作內容
需求分析	確定功能範圍與技術選型
技術研究	測試 MediaPipe，確保技術可行
模型訓練	訓練並調整手語辨識模型
前端開發	設計 UI/UX，開發網頁介面
系統整合	將辨識模型與網站前端整合
測試與優化	修正錯誤，提升辨識準確率與使用體驗
報告撰寫	撰寫專題報告與簡報

△表 2-1-1 時間可行性 (Schedule Feasibility)

結論：本專案可按照開發階段逐步完成，團隊可根據進度適當調整時程。

2-2 商業模式—Business model

關鍵合作夥伴：特殊教育學校、手語協會、政府單位、AI 技術合作方	關鍵活動：開發 AI 模型、優化手語教學內容、行銷推廣	價值主張：提供即時手語翻譯與學習，讓聽障者與社會溝通更便利	顧客關係：提供免費試用、用戶回饋調查	目標客群：主要為聽人、聽障人士、手語學習者、教育機構、企業與政府機構
	關鍵資源：AI 手語辨識技術、手語課程內容、開發團隊		通路：官方網站、社群媒體（Facebook、YouTube）、學校合作	
成本結構：開發成本、伺服器成本（若未來使用雲端）、行銷費用		收益流：訂閱制、廣告、企業合作、API 授權		

△表 2-2-1 商業模式—Business model

2-3 STP 分析

市場區隔	目標市場	產品定位
1.聽障人士-需要即時手語翻譯來與不懂手語者溝通 2.手語學習者-希望透過互動式 AI 系統提升手語學習效果 3.教師與教育機構-需要輔助手語教學的數位工具	主要針對聽人，同時擴展至手語學習者、教師與教育機構，提升學習與溝通效率。	1.高準確度的 AI 手語辨識系統，支援即時手語翻譯，提高聽障人士與聽人之間的溝通效率。 2.互動式手語學習工具，提供即時回饋與錯誤修正機制，提升學習效果。 3.適用於教育與無障礙應用場景，可用於學校、醫療機構、客服等多種情境，幫助更多使用者。

△表 2-3-1 STP 分析

2-4 競爭力分析 SWOT-TOWS

優勢(Strengths)

- 技術創新，整合即時手語翻譯與教學功能，提供雙向溝通與學習體驗。
- 多功能性，涵蓋翻譯與手語練習教學，滿足多樣需求。
- 有社會價值，推動無障礙溝通與多元共融，呼應 ESG 趨勢。

劣勢(Weaknesses)

- 手語識別準確率需持續優化，技術難度與訓練成本高。
- 開發與維護成本高、耗時長，需投入大量資源。
- 使用者對 AI 操作的熟悉度參差不齊，需良好的引導與介面設計。

機會(Opportunities)

- 政府或企業有資助無障礙技術的政策與補助資源。
- 市場上對手語學習與翻譯需求提升，教育與醫療領域皆有應用潛力。
- 可與校園、醫院、公益機構等建立合作機會。

威脅(Threats)

- 科技大廠如 Google、Microsoft 已投入手語辨識市場。
- 各國手語差異大，增加系統跨區域應用的挑戰。
- 翻譯結果不穩定時會影響使用者信任與黏著度。

	優勢	劣勢
機會	1.主動申請政府「科技輔具」與「無障礙資訊科技」相關補助計畫，並結合 ESG 永續發展理念，以爭取企業 CSR 資金合作與支持。 2.聯合特殊教育學校與聽語障協會進行實地測試，建立公信力與真實案例。 3.導入 AI 手語教學小遊戲，結合國內外語言學習平台做 API 整合拓展受眾。	1.與 AI 模型開源社群（如 HuggingFace、TensorFlow）合作，加速訓練模型與修正錯誤。 2.與大學研究機構（如資訊管理、特教系）合作，委託專題或論文共同研發，降低人力成本。 3.透過國發基金或創新育成的輔導，加快產品 MVP 推出時程。
威脅	1.開發「在地手語語系模型」，專注台灣手語（TSS），在區域市場建立領導地位。 2.加入「教育部輔具資源中心」、「身心障礙資源平台」等生態系，提高曝光與信任度。 3.強化翻譯結果可信度：在每次翻譯後附上使用信心指標（信心度 %），提升使用者體驗。	1.先推出「手語教學」功能為主的教育版本 App（成本低、難度低），慢慢擴展到「即時翻譯」。 2.設計引導式 UI/UX：提供語音教學、互動式操作導覽，降低新手流失率。 3.設置用戶回饋機制，快速迭代版本，避免因錯誤率高而失去信任。

△表 2-4-1 競爭力分析 SWOT-TOWS

第三章 系統規格

3-1 系統架構

下圖為本專題前後端應用工具、框架及系統架構

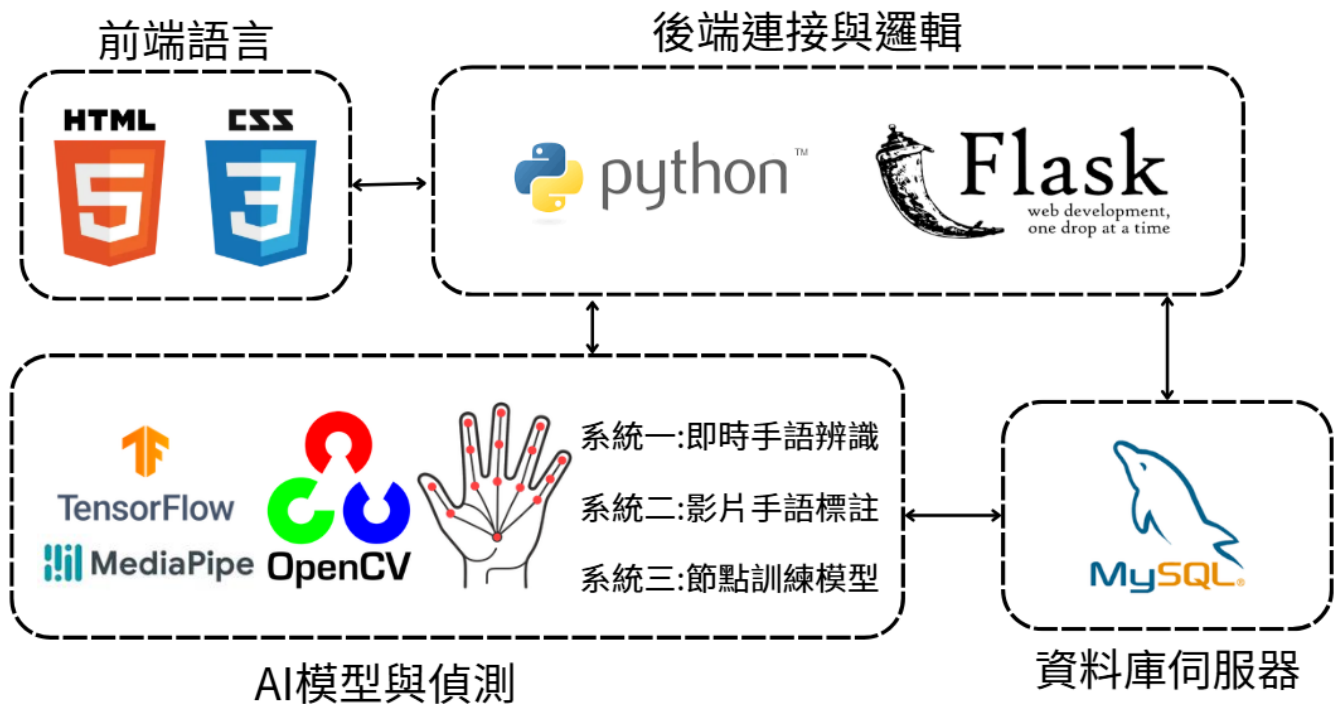


圖 3-1-1 系統架構

1. 系統一:即時手語辨識

採用「攝影機影格擷取 → 關鍵點偵測與特徵整形 → 固定窗序列打包(120 幀) → Transformer(Keras)分類 → 畫面顯示與穩定機制」的單向循環，以即時偵測與即時推論為核心架構。系統以 OpenCV 擷取攝影機每幀影像，前端以 MediaPipe Hands + Face Mesh 同步進行雙手與臉部偵測；每幀萃取雙手 21 點 $\times(x,y,z)\times 2$ 與臉部 68 點 $\times(x,y,z)$ ，合併為 330 維特徵，並先以「臉部中心」作為座標原點，將手與臉關鍵點座標統一做相對平移，以降低人物在畫面中位置變化造成的偏移影響；接著再對每幀特徵做 (x,y,z) 維度的標準化，使不同幀的尺度更一致。序列端採用固定窗機制：維持長度為 120 的 buffer，當 buffer 滿 120 幀時才送入模型推論。推論模型

為 TensorFlow/Keras 載入的 Transformer 模型 (.h5)，並以自訂 PositionalEncoding 層支援模型讀取。顯示端加入穩定機制：若畫面中沒有偵測到手則直接顯示「搜尋中」並清空 buffer，避免殘留舊手勢導致誤判；若有高於信心門檻(0.8)的辨識結果，則記錄最近一次成功標籤與信心值，並以固定顯示秒數(2.5s)維持畫面文字，期間同時清空 buffer 重新蒐集下一輪手勢，以降低「上一個手勢延遲出現」的體感問題；同時在畫面上繪製手部與臉部網格以利操作觀察。

2. 系統二:影片手語標註

使用的套件與工具:

- OpenCV：影片讀寫、影像處理、GUI 顯示與鍵盤滑鼠事件。
- MediaPipe Face Mesh：臉部偵測與 68 點臉部關鍵點回歸
- MediaPipe Hands：左右手 21 點關鍵點與慣用手推論（最多 2 隻手）
- NumPy / CSV：特徵整形、標準化與 JSON/CSV 輸出。

A. 節點紀錄

本系統的處理流程如下：首先載入影片與模型，讀取選取的檔案後，初始化 MediaPipe Face Mesh 臉部偵測器與 68 點關鍵點預測器，以及 MediaPipe Hands。接著進行逐幀處理：對臉部先做偵測再回歸 68 個關鍵點，取得 (x, y) 並以 0 補上 z；同時以 MediaPipe 偵測雙手並辨識左右手，將其正規化座標依畫面寬高換算為畫素座標並分配至 L/R。畫面上即時可視化，右側資訊欄可捲動查看臉與雙手座標，並同步顯示播放進度。每一幀則依固定欄序將右手 21×(x,y,z)、左手 21×(x,y,z) 與臉部 68×(x,y,z)共 110×3=330 個點，攤平成單列暫存於記憶體。最後輸出 coordinates.csv 檔案，欄序=frame+R0_x,R0_y,R0_z,...,R20_*+L0_*...L20_*+F1_*...F68_* (共 1

+ 21×3 + 21×3 + 68×3 欄)。

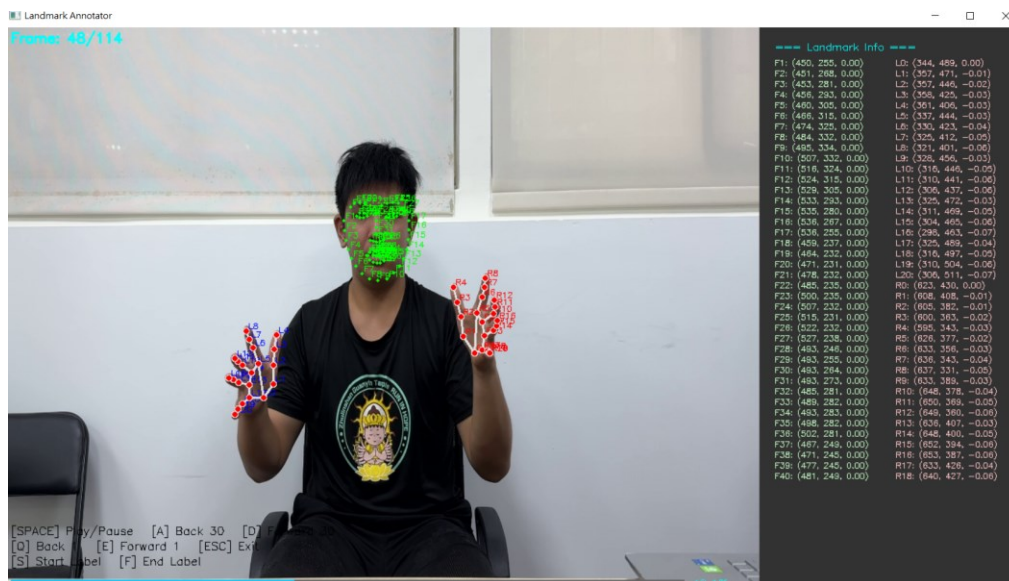


圖 3-1-2 可視化節點辨識紀錄

圖 3-1-3 節點 CSV 檔範例

B. 標註流程

以 Tkinter 建構圖形介面，整合影片載入、時間軸與幀數顯示、以及標註清單管理；輸入標籤文字後，透過「設定起點→播放至目標→設定終點並儲存」的操作即可完成一筆區段標註。系統同時以 OpenCV 讀取影格、取得 FPS 與總幀數，並支援拖曳式跳轉；畫面更新過程中再以 Pillow 將 OpenCV 影像轉為 Tkinter 可顯示的物件。每次完成標註，工具便將「影片名稱、起點幀、終點幀、標籤」四欄資料

追加寫入對應的 annotations/<影片名>.csv

完成標註後，承接前述輸出並與逐幀關鍵點座標檔 coordinates.csv 對齊處理。先以檔案選擇讀入座標與標註兩份 CSV，再依每筆標註的 start_frame 與 end_frame 篩出對應幀範圍的資料。以 R（右手）、L（左手）、F（臉部）為前綴的關鍵點欄位，並將各幀展平成一致的數列格式後組裝為段落級 JSON；每一段輸出一個檔案，內含 label、video、start_frame、end_frame 與 frames[{frame, keypoints[]}]欄位。



圖 3-1-4 影片標註

```

1  {
2    "label": "不見",
3    "video": "不見9",
4    "start_frame": 0,
5    "end_frame": 113,
6    "frames": [
7      {
8        "frame": 0,
9        "keypoints": [
10         0.0,
11         0.0,
12         0.0,
13         0.0,
14         0.0,
15         0.0,
16         0.0,
17         0.0,
18         0.0

```

圖 3-1-5 JSON 檔格式

C. 後續節點紀錄與標註程序優化

本系統的處理流程如下：首先掃描指定資料夾內所有影片檔 (mp4/mov/avi)，逐一讀取影片並進行逐幀處理。每幀先以 Face Mesh 取得臉部 68 點 (x,y,z)，並計算其平均作為 face_center；再以 Hands 偵測最多兩隻手，各手 21 點 (x,y,z) 會先減去 face_center 轉為相對座標，以統一人物在畫面中的位移差異。手部特徵採固定長度設計：最多收兩隻手，若偵測不足則以 0 補齊，確保每幀手部維度恆為 $21 \times 3 \times 2$ 。臉部若未偵測到則以 0 向量補齊，確保臉部維度固定為 68×3 。接著將「雙手+臉部」合併成 330 維向量後，再將其 reshape 成 (-1,3)，對 (x,y,z) 三個維度做標準化 (mean/std)，使每幀特徵在尺度上更一致。最後將整支影片的 frames 以固定長度 120 幀輸出：不足則以全 0 幀補齊，超過則截斷前 120 幀，並輸出為 JSON 檔案，格式為 {"label": 影片檔名(去副檔名), "frames": [120×330 的序列]}，作為後續訓練/推論共用的序列資料來源。

目前版本的資料建置改為「以檔名作為類別標籤」的自動化流程，不再以區段

式時間標註為主，而是將每支影片視為一筆樣本直接產生對應 JSON。後續前處理會讀入所有 JSON，並以檔名/JSON 內 label 作為類別來源，同時移除檔名中的數字（例如「早 1」「早 2」會歸併為「早」）以提升同義樣本的一致性；此外會逐幀檢查每幀長度是否為 330，若異常則自動補零或截斷修正，並統一補齊/截斷為 120 幀。最後輸出 dataset_3d.npz（含 X_train/X_test/y_train/y_test）、label_map.npy（類別索引對照表），並同時輸出 train_list.txt 與 test_list.txt 以保留可追溯的切分名單。

3. 系統三:節點訓練模型

以「固定長度關鍵點序列」為核心資料型態，建置一套從資料載入、模型訓練到推論部署一致的流程。資料端直接讀入 dataset_3d.npz（若存在 dataset_3d_augmented.npz 則優先使用擴增後資料），其輸入張量形狀為 [N, 120, 330]；其中 120 為固定幀數，330 為每幀特徵（雙手+臉 68 點）。模型定義為 Keras 的序列分類器：輸入先疊加不可學習的正弦位置編碼（PositionalEncoding），讓模型具備時序感知；接著堆疊兩層 Transformer block，每層包含 Multi-Head Self-Attention、Dropout 與殘差 LayerNorm，以及前饋網路（Dense-ReLU-Dense）再配合殘差 LayerNorm，以捕捉序列內的長短期依存關係。序列彙聚採用 GlobalAveragePooling1D 將時間維壓縮，再接全連接層(256)與 Dropout(0.3)，最後以 softmax 輸出各類別機率。。

訓練流程使用 Adam(lr=1e-4) 與 sparse_categorical_crossentropy，批次大小 8、最多 80 epochs，並以三種 callbacks 強化訓練穩定性與可用性：EarlyStopping 監控 val_accuracy 並在耐心值內無提升時停止訓練且回復最佳權重；ModelCheckpoint 於 val_accuracy 創新高時輸出最佳整體模型 transformer_3d_best.h5；

ReduceLROnPlateau 於 val_loss 停滯時自動降低學習率，使訓練更容易收斂。訓練完成後輸出的 .h5 模型可直接被即時辨識端載入，並透過 custom_object_scope 註冊 PositionalEncoding 以確保模型可正確反序列化與推論。

```
563/563 [=====] - ETA: 0s - loss: 0.0033 - accuracy: 1.0000
Epoch 8: val_accuracy did not improve from 0.98658
563/563 [=====] - 125s 222ms/step - loss: 0.0033 - accuracy: 1.0000 - val_loss: 0.0541 - val_accuracy: 0.9866 - lr: 1.0000e-04
Epoch 9/80
563/563 [=====] - ETA: 0s - loss: 0.0020 - accuracy: 1.0000
Epoch 9: val_accuracy did not improve from 0.98658
563/563 [=====] - 122s 217ms/step - loss: 0.0020 - accuracy: 1.0000 - val_loss: 0.0527 - val_accuracy: 0.9866 - lr: 1.0000e-04
Epoch 10/80
563/563 [=====] - ETA: 0s - loss: 0.0014 - accuracy: 1.0000
Epoch 10: val_accuracy did not improve from 0.98658
563/563 [=====] - 132s 235ms/step - loss: 0.0014 - accuracy: 1.0000 - val_loss: 0.0472 - val_accuracy: 0.9866 - lr: 1.0000e-04
Epoch 11/80
563/563 [=====] - ETA: 0s - loss: 0.0568 - accuracy: 0.9836
Epoch 11: val_accuracy did not improve from 0.98658
563/563 [=====] - 150s 266ms/step - loss: 0.0568 - accuracy: 0.9836 - val_loss: 0.0667 - val_accuracy: 0.9866 - lr: 1.0000e-04
Epoch 12/80
563/563 [=====] - ETA: 0s - loss: 0.0103 - accuracy: 0.9984
Epoch 12: val_accuracy did not improve from 0.98658
563/563 [=====] - 153s 272ms/step - loss: 0.0103 - accuracy: 0.9984 - val_loss: 0.0807 - val_accuracy: 0.9866 - lr: 1.0000e-04
Epoch 13/80
563/563 [=====] - ETA: 0s - loss: 0.0058 - accuracy: 0.9987
Epoch 13: val_accuracy did not improve from 0.98658
563/563 [=====] - 155s 275ms/step - loss: 0.0058 - accuracy: 0.9987 - val_loss: 0.0618 - val_accuracy: 0.9866 - lr: 1.0000e-04
Epoch 14/80
563/563 [=====] - ETA: 0s - loss: 8.6567e-04 - accuracy: 1.0000
Epoch 14: val_accuracy did not improve from 0.98658
563/563 [=====] - 142s 251ms/step - loss: 8.6567e-04 - accuracy: 1.0000 - val_loss: 0.0615 - val_accuracy: 0.9866 - lr: 1.0000e-04
Epoch 15/80
563/563 [=====] - ETA: 0s - loss: 6.5186e-04 - accuracy: 1.0000
Epoch 15: val_accuracy did not improve from 0.98658

Epoch 15: ReduceLROnPlateau reducing learning rate to 4.99999873689376e-05.
563/563 [=====] - 141s 250ms/step - loss: 6.5186e-04 - accuracy: 1.0000 - val_loss: 0.0567 - val_accuracy: 0.9866 - lr: 1.0000e-04

🔥 訓練完成！最好的模型已儲存至：models/transformer_3d_best.h5
```

圖 3-1-6 訓練結果

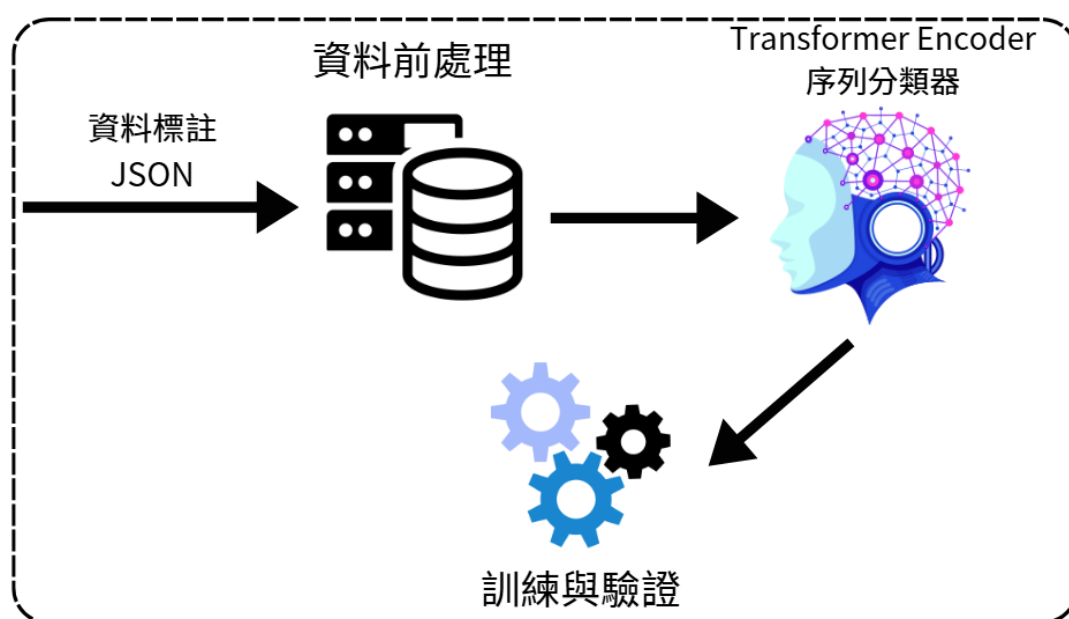


圖 3-1-7 訓練流程概念圖

3-2 系統軟、硬體需求與技術平台

持備有 webcam 的筆電或桌上型電腦設備並連上網路即可連接到本網站。

3-3 使用標準與工具

系統開發環境	
作業系統	Windows 10
程式開發工具	
前端	HTML、CSS
後端	Python、flask
資料庫	MySQL
模型	MediaPipe、OpenCV、TensorFlow/Keras
專案管理平台	
專案管理	Github、fork
檔案存放	Github、google drive
文件撰寫工具	
文件製作	Microsoft Word
簡報製作	Microsoft PowerPoint、Canva
UML	Visual Paradigm

表 3-3-1 使用標準與工具

第四章 專案時程與組織分工

4-1 專案時程

實際		預計	
----	--	----	--

表 4-1-1 專案時程

114 年										
	1 月	2 月	3 月	4 月	5 月	6 月	7 月	8 月	9 月	10 月
專題構想										
需求分析										
後端撰寫										
資料庫建置										
文件製作										
資料蒐集										
資料處理										
影片拍攝										
模型訓練										
介面設計										
前端撰寫										
系統整合										
系統測試										
簡報製作										

4-2 專案組織與分工

●主要負責人 ○次要負責人

項目/組員		11046042/ 周駿翔	11146038/ 郭子寧	11146003/ 黃俊翰	11146040/ 黃政
後 端 開 發	資料庫建置		○		●
	伺服器架設				●
	影片切割	●		○	
	模型訓練	●	○		
	手部節點辨識			●	○
	臉部節點辨識			○	●
	影片拍攝	●	●	●	●
	資料集建設	●		○	
	資料前處理	●	○		
	節點標註	●			
前 端 開 發	首頁		●		
	登入、登出		●		
	註冊		●		
	學習手語		●		
	影片播放		●		
	使用者資料		●		
	即時手語翻譯		●		

	關於我們		●		
	學習手語		●		
美術設計	UI/ UX		●		
	Web/APP 介面設計		●		
	色彩設計		●		
	Logo 設計	●			
文件撰寫	統整	●			
	第 1 章 前言	●		○	
	第 2 章 營運計畫	●	○		○
	第 3 章 系統規格	●		○	○
	第 4 章 專題時程與組織分工	●			
	第 5 章 需求模型	●			
	第 6 章 <u>設計</u> 模型	●			
	第 7 章 <u>實作</u> 模型				●
	第 8 章 資料庫設計				●
	第 9 章 程式			●	
	第 10 章 測試模型				●
	第 11 章 操作手冊				●
	第 12 章 使用手冊		●		
報	簡報製作	●	○		○

告					
---	--	--	--	--	--

表 4-2-1 專案組織與分工

序號	姓名	工作內容	貢獻度
1	組長 周駿 翔	參與專案主題與方向訂定，製作文件，負責協調所有事物，主要負責手語辨識模型訓練，節點標註，協助影片資料蒐集，以及影片分割。	<u>30%</u>
2	組員 郭子 寧	參與專案主題與方向訂定，主要負責網頁介面設計與前後端功能，並協助資料庫結構規劃與管理、模型訓練、影片拍攝等。	<u>30%</u>
3	組員 黃俊 翰	參與專案主題與方向訂定，主要負責前期手語辨識研究、臉部及手部節點紀錄，以及手語影片資料拍攝，	<u>20%</u>
4	組員 黃政	參與專案主題與方向訂定，製作文件，主要負責資料庫結構規劃與管理、影片拍攝、協助前期臉、手部節點紀錄。	<u>20%</u>

表 4-2-2 工作內容與貢獻度

4-3 上傳 GitHub 紀錄

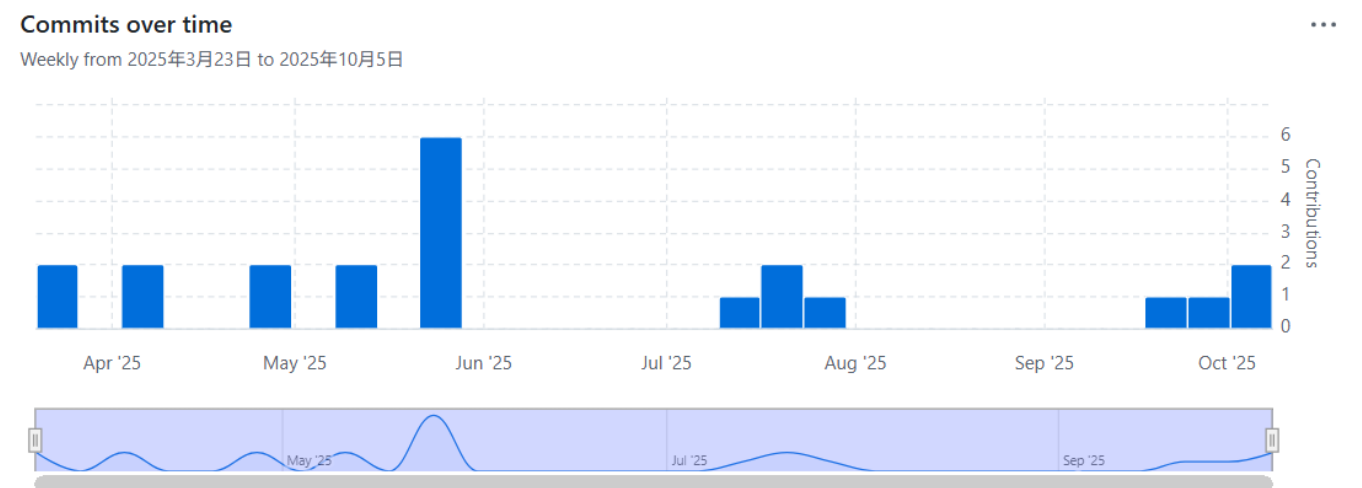


圖 4-3-1 總體上傳

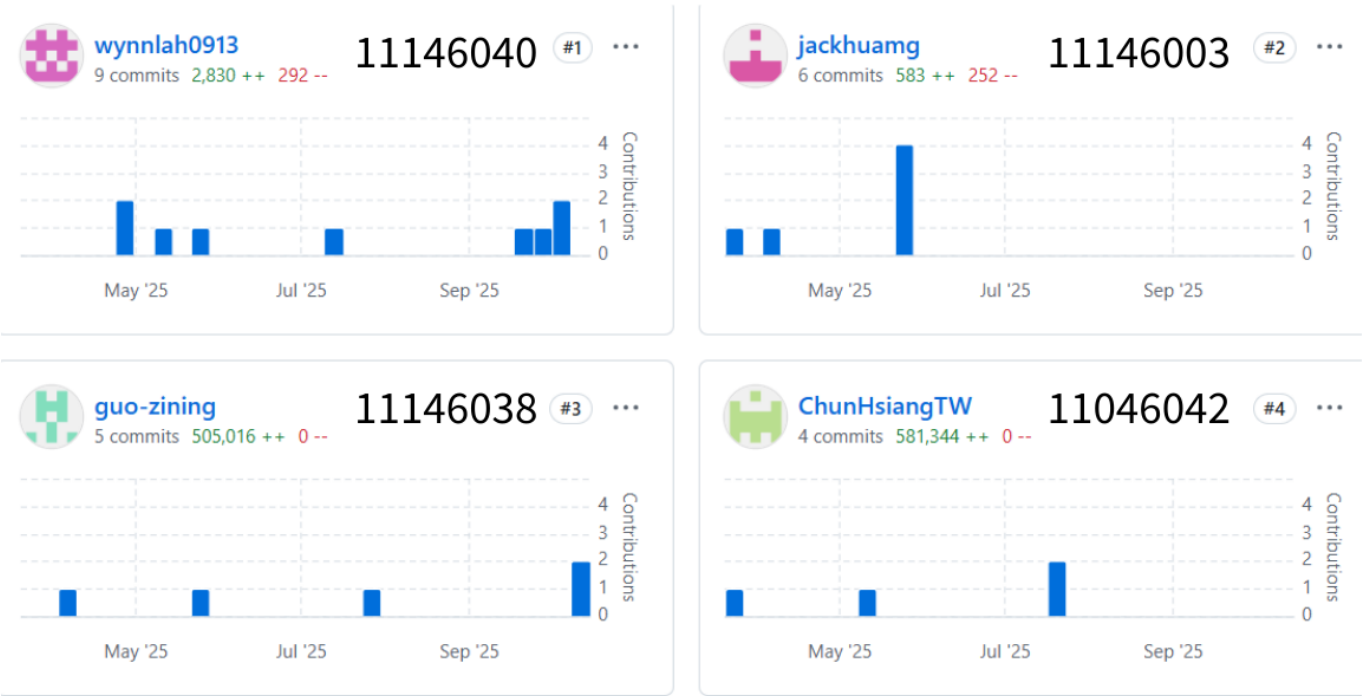


圖 4-3-2 個別上傳

第五章 需求模型

5-1 使用者需求

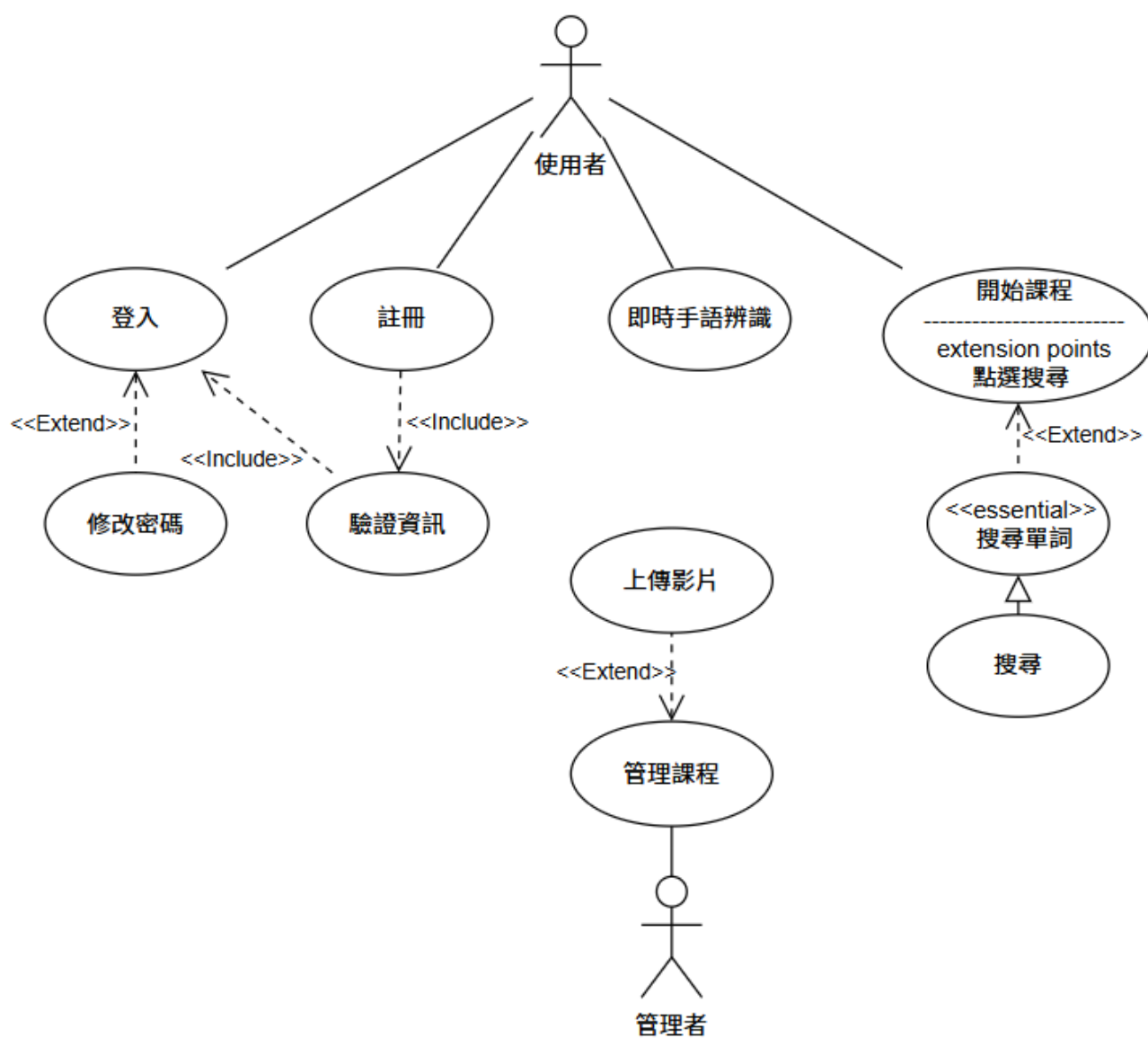
功能性需求	說明
登入與註冊	使用者透過電子郵件登入註冊網站
即時手語辨識	透過鏡頭捕捉使用者手部姿勢並翻譯
練習	使用者透過辨識系統自行練習手語辭彙
課程	使用者可觀看教學影片
搜尋	使用者可依照關鍵字/類別搜尋單詞或課程

△表 5-1-1 功能性需求

非功能性需求	說明
簡潔的頁面	更方便的使用系統功能
設計成網頁形式	無須下載軟體，只需開起網站即可
學習紀錄自動儲存	使用者課程進度與測驗紀錄自動儲存，避免進度遺失

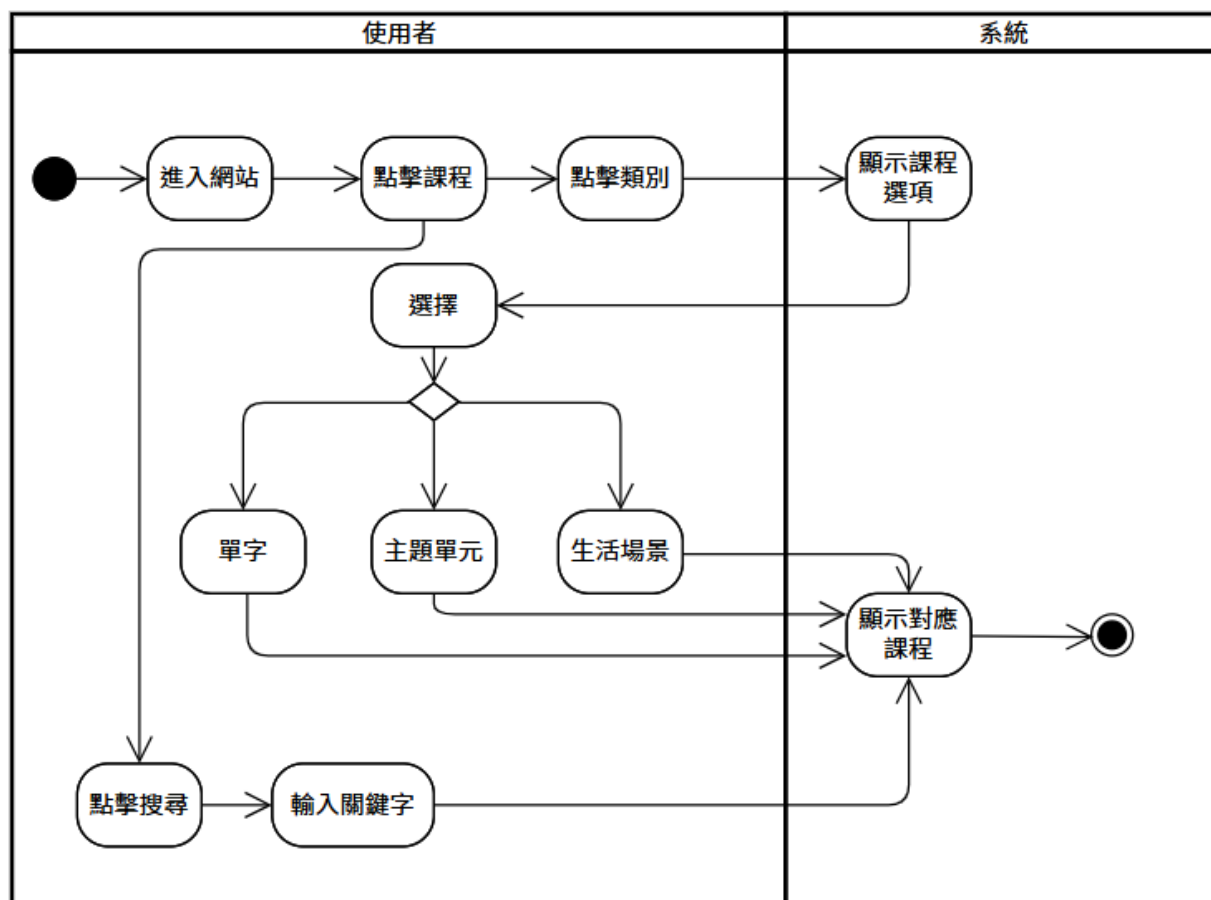
△表 5-1-2 非功能性需求

5-2 使用個案圖(Use case diagram)

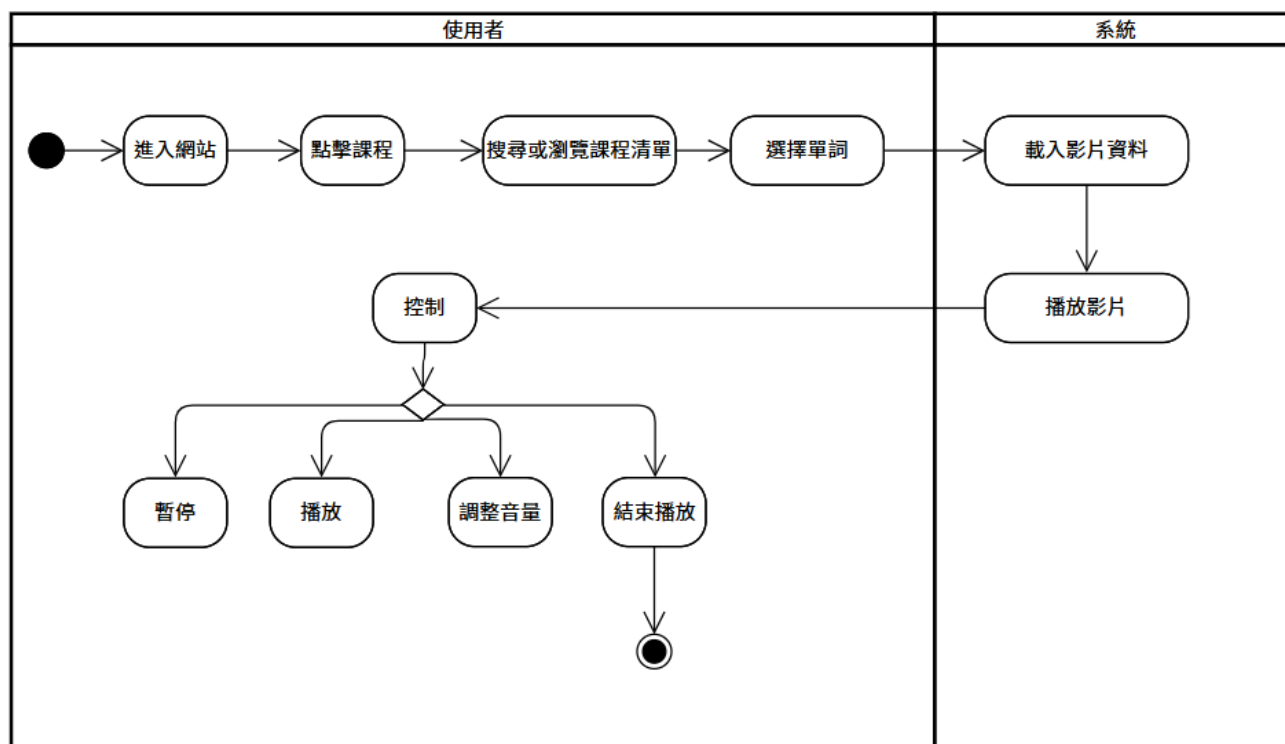


△圖 5-2-1 使用個案圖

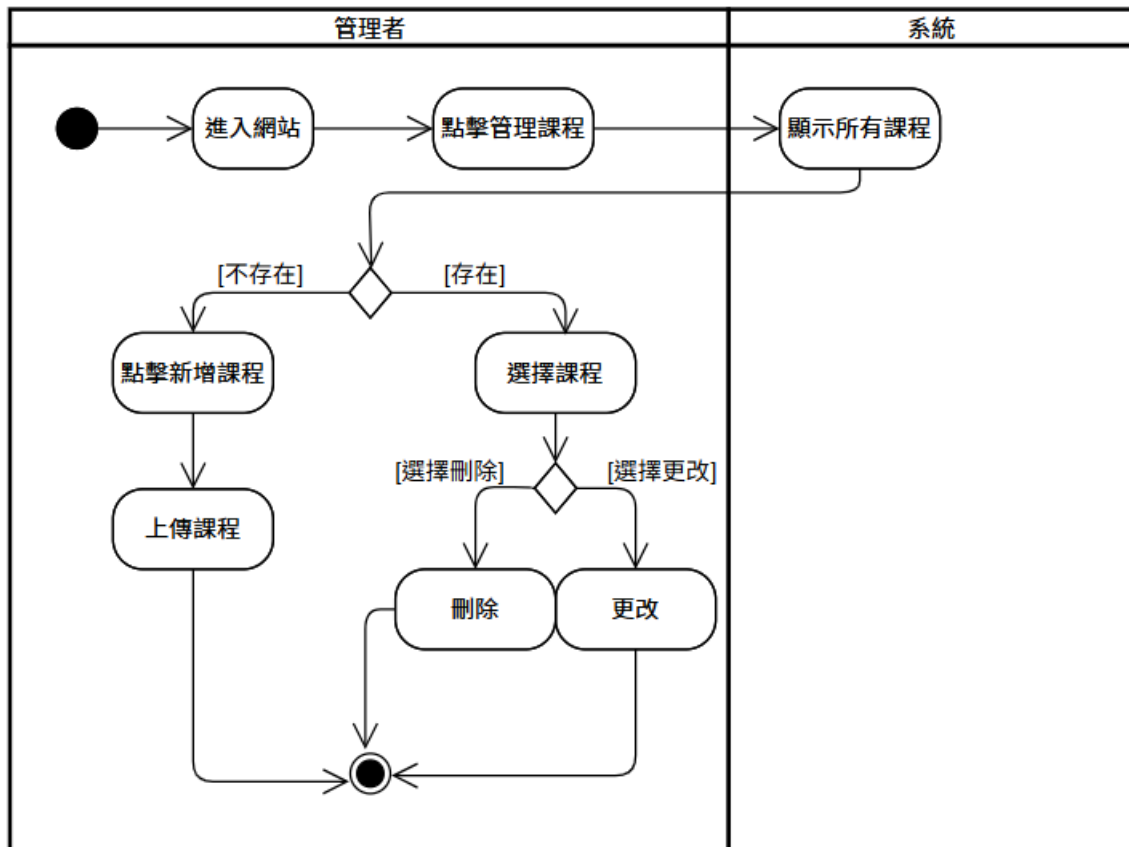
5-3 使用個案描述



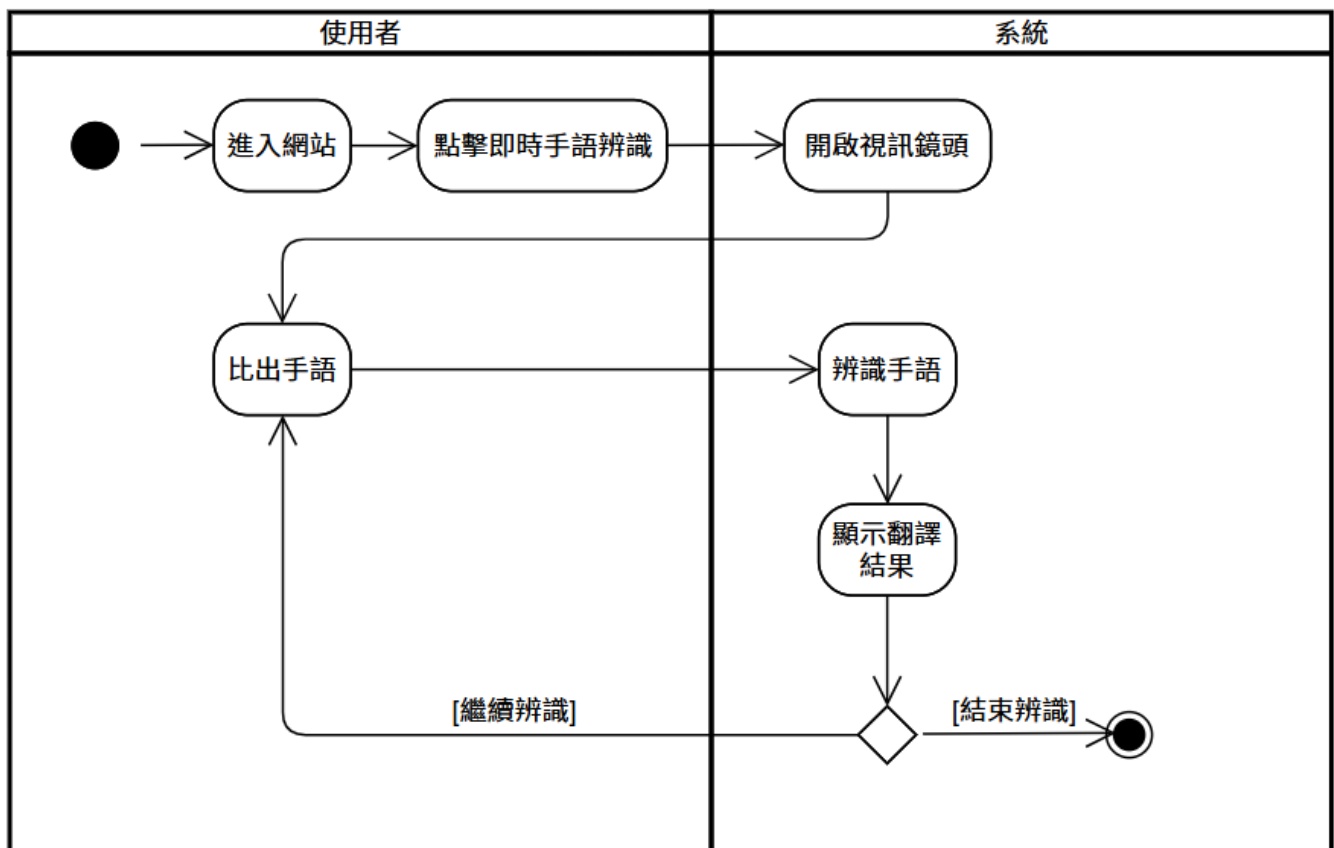
△圖 5-3-1 搜尋之活動圖



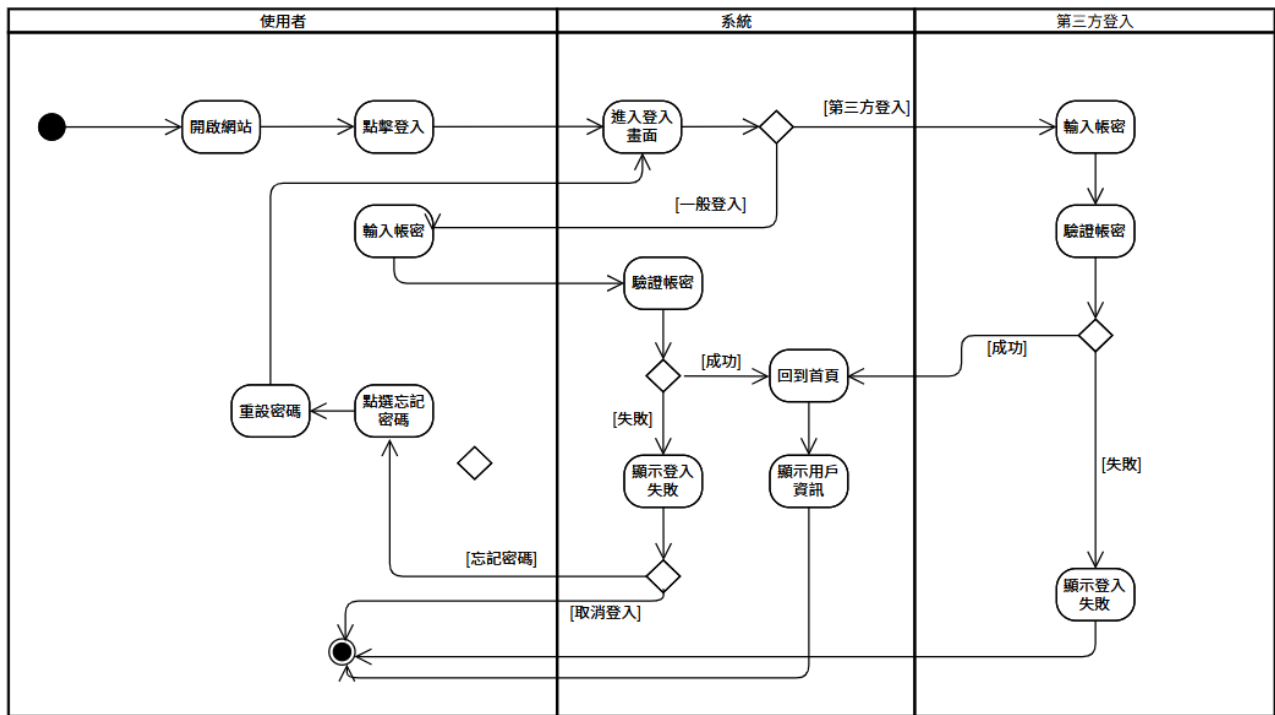
△圖 5-3-2 課程觀看之活動圖



△圖 5-3-3 管理課程之活動圖

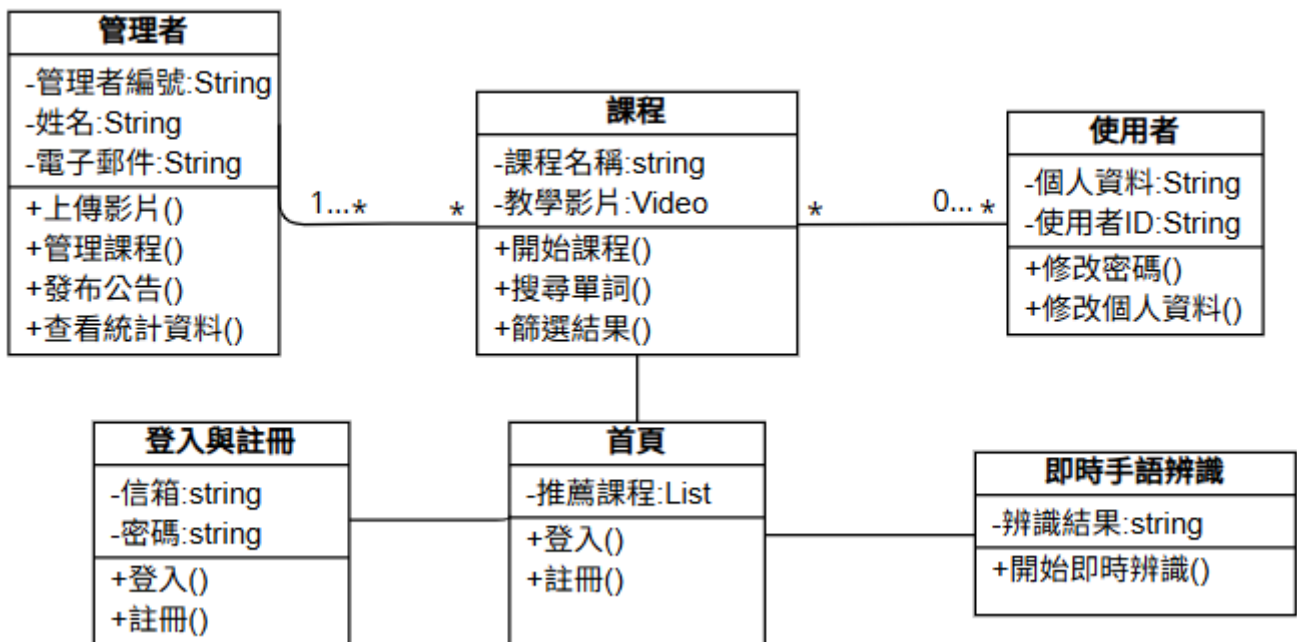


△圖 5-3-4 即時辨識之活動圖



△圖 5-3-5 登入之活動圖

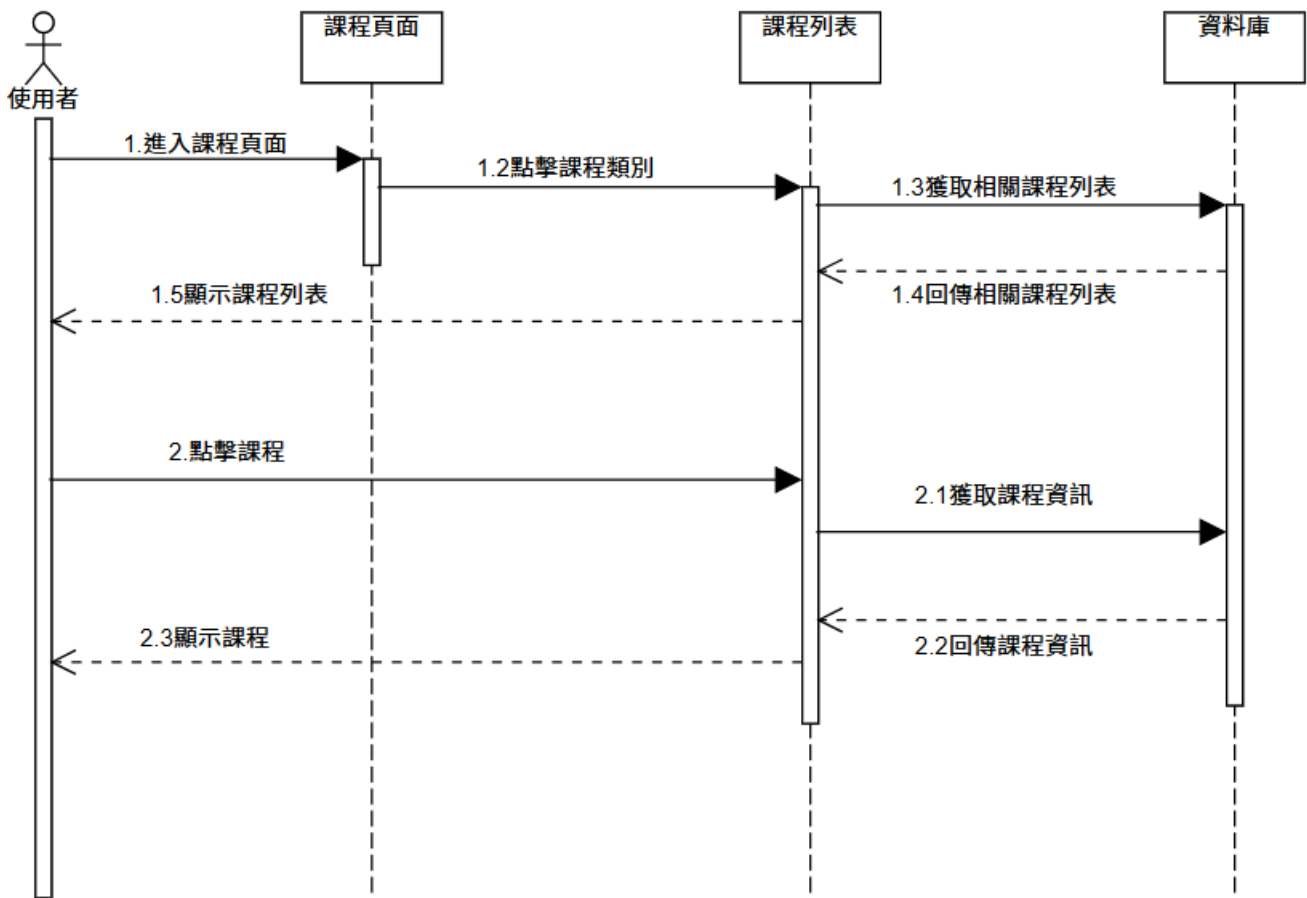
5-4 分析類別圖(Analysis class diagram)



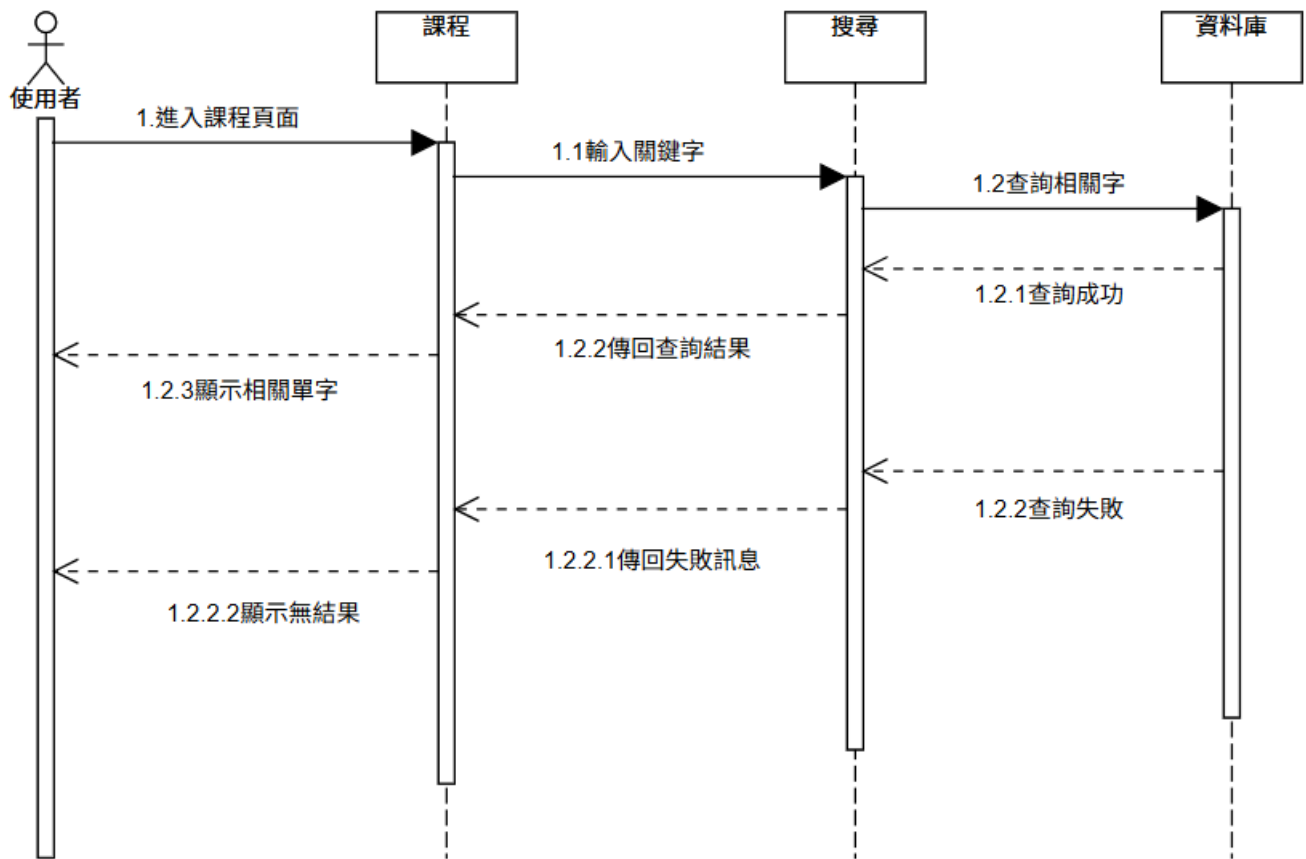
△圖 5-4-1 分析類別圖(Analysis class diagram)

第六章 設計模型

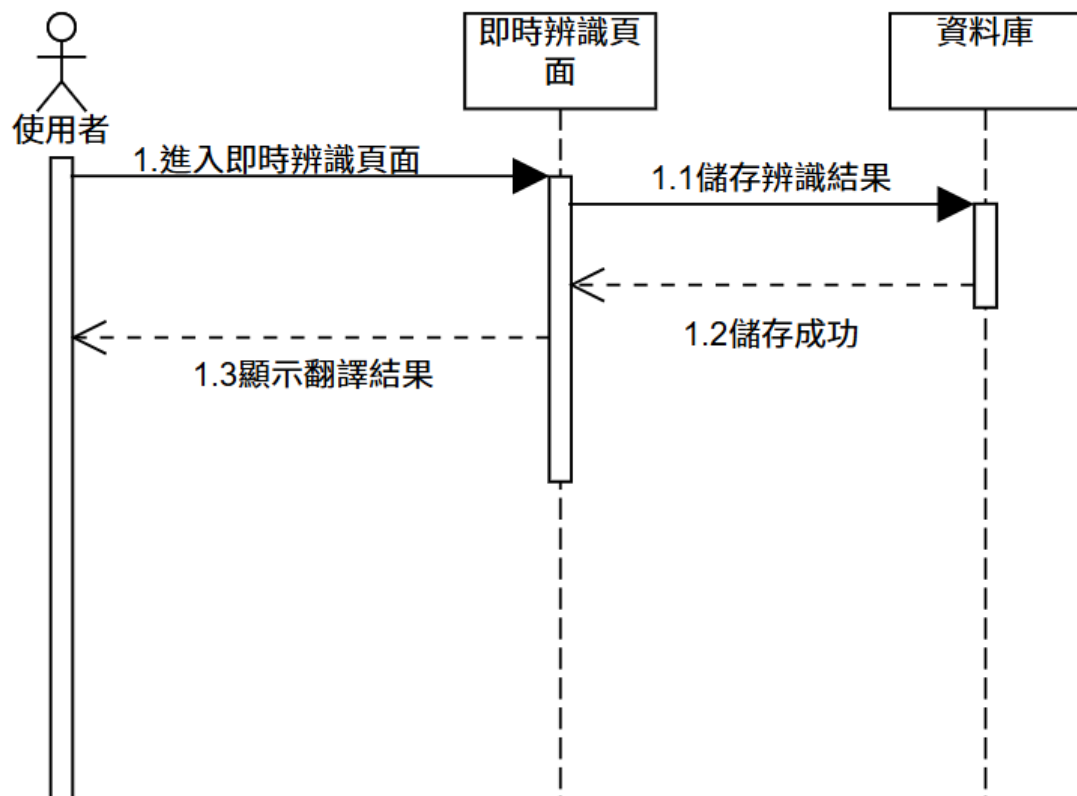
6-1 循序圖(Sequential diagram)



△圖 6-1-1 查看課程之循序圖

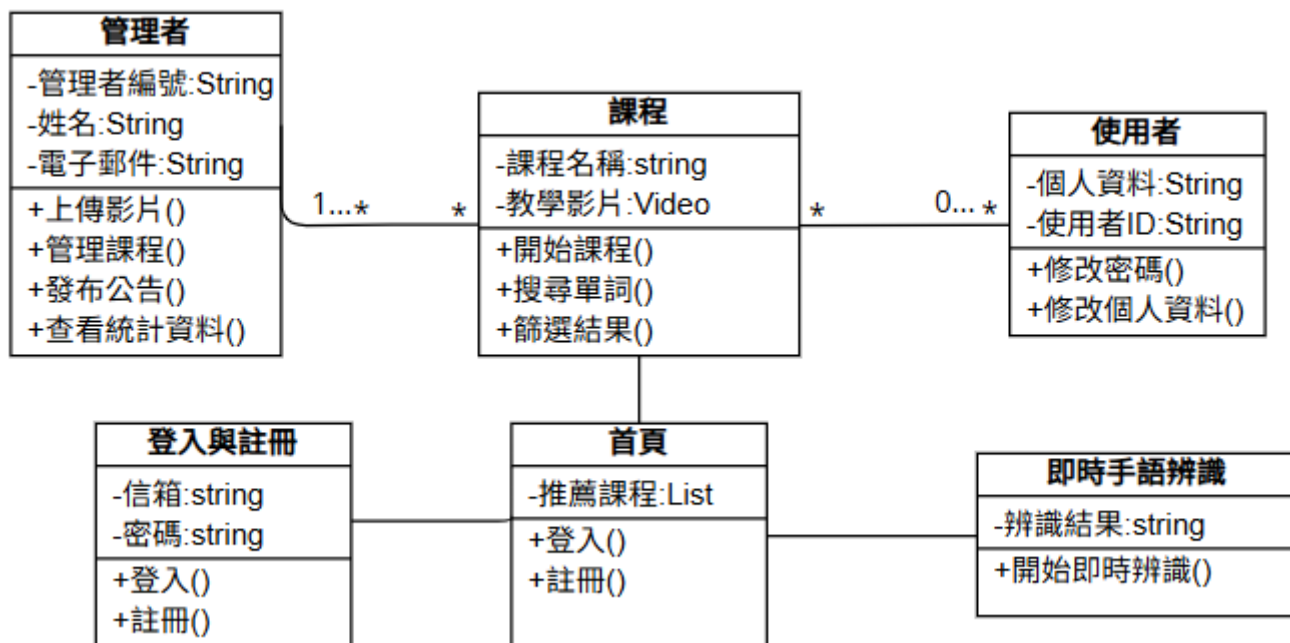


△圖 6-1-2 搜尋之循序圖



△圖 6-1-3 開啟即時辨識之循序圖

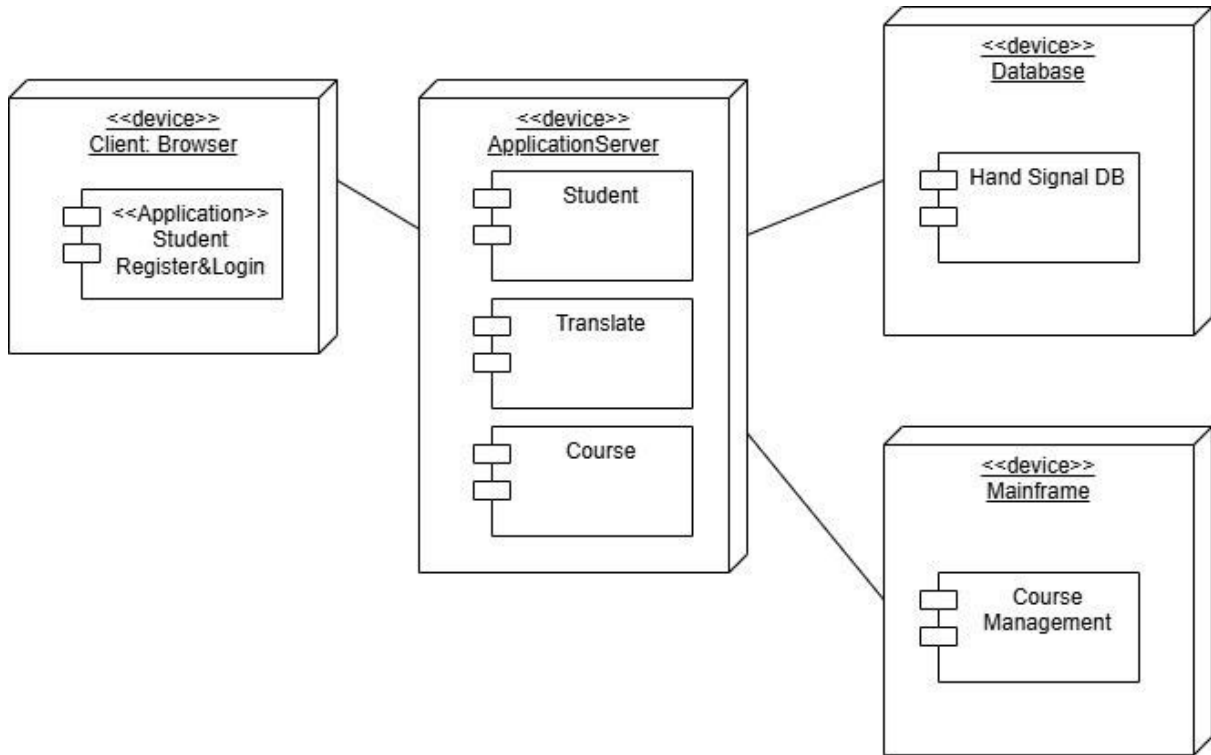
6-2 設計類別圖(Design class diagram)



△圖 6-2-1 設計類別圖(Design class diagram)

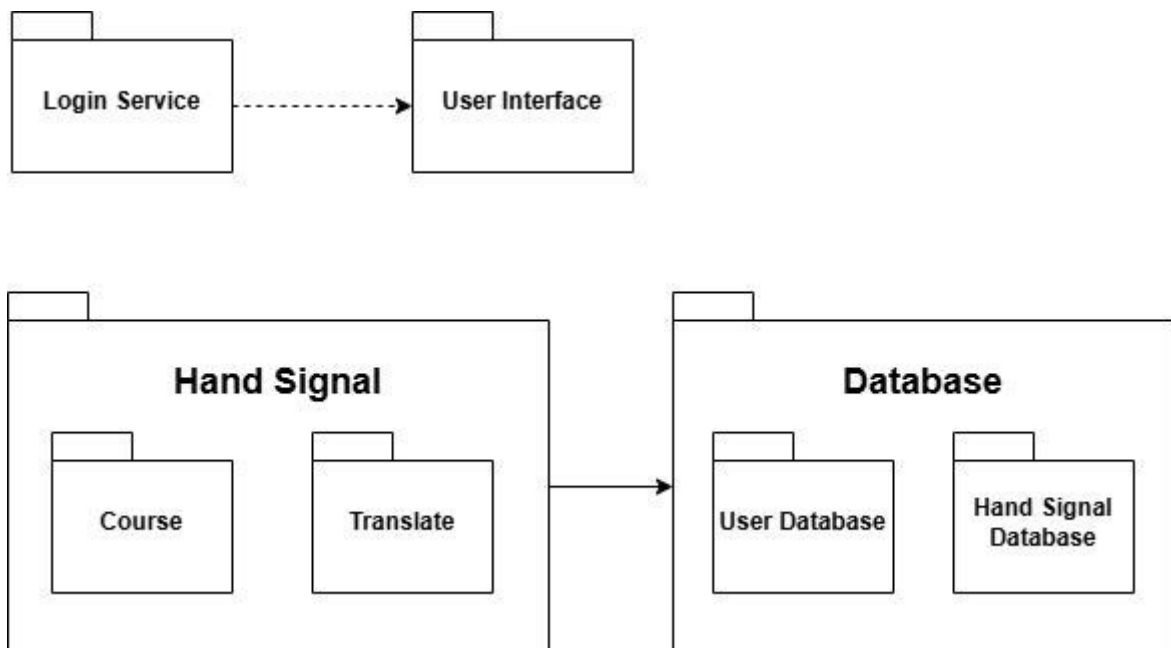
第七章 實作模型

7-1 佈署圖(Deployment diagram)



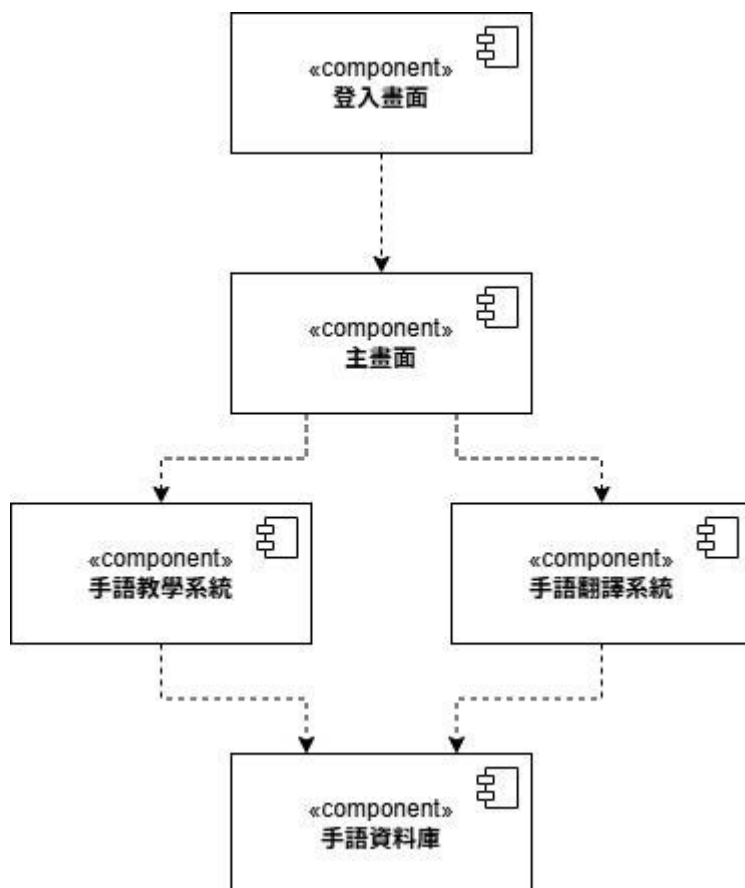
△圖 7-1-1 佈署圖(Deployment diagram)

7-2 套件圖(Package diagram)



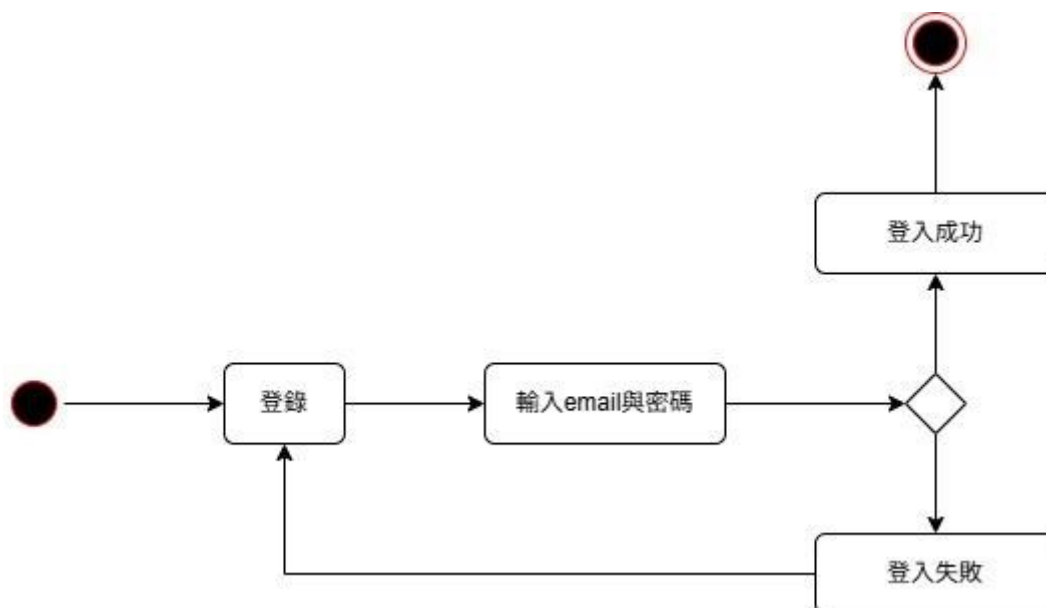
△圖 7-2-1 套件圖(Package diagram)

7-3 元件圖(Component diagram)



△圖 7-3-1 元件圖(Component diagram)

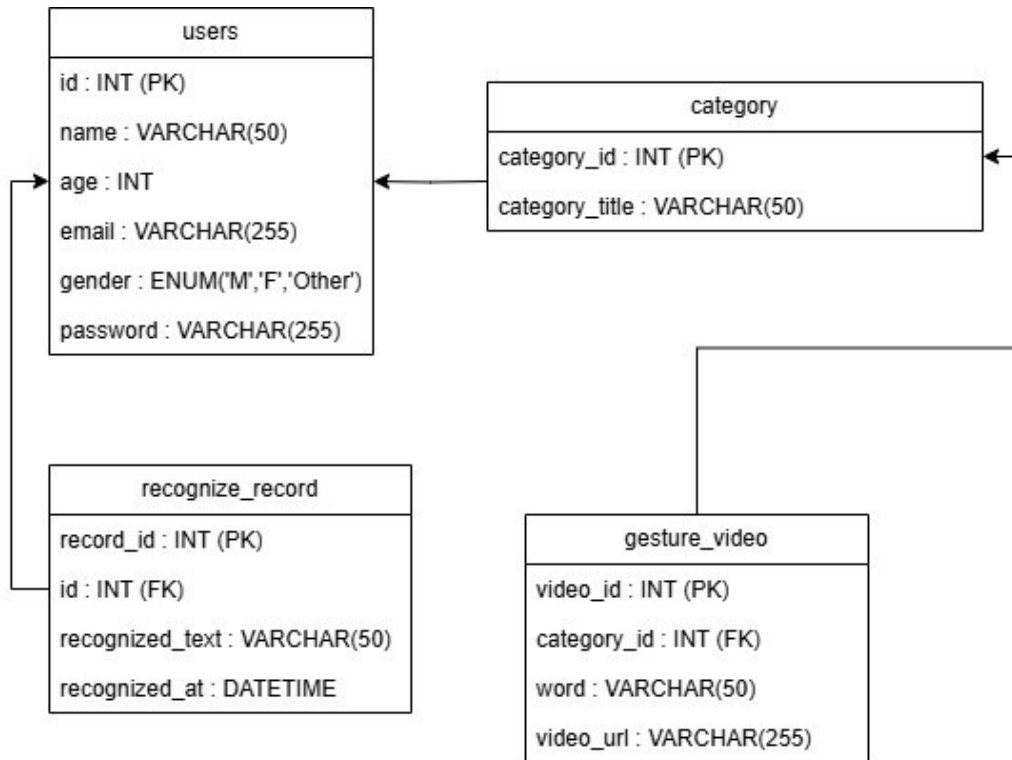
7-4 狀態機(State machine)



△圖 7-4-1 狀態機(State machine)

第八章 資料庫設計

8-1 資料庫關聯表



△圖 8-1-1 資料庫關聯表

8-2 表格及其 Meta data

users				
名稱	中文名稱	資料形態	資料長度	索引
id	用戶流水號	int	-	PK
name	用戶名	varchar	50	
age	年齡	int	-	
email	電子郵件	varchar	255	
gender	性別	enum	-	
password	密碼	varchar	255	

△表 8-2-1 users 之表格

category				
名稱	中文名稱	資料形態	資料長度	索引
category_id	類別流水號	int	-	PK
category_title	類別標題	varchar	50	

△表 8-2-2 category 之表格

gesture_video				
名稱	中文名稱	資料形態	資料長度	索引
video_id	影片流水號	int	-	PK
category_id	類別流水號	int	-	FK
word	詞匯	varchar	50	
video_url	影片位址	varchar	255	

△表 8-2-3 gesture_video 之表格

recognize_record				
名稱	中文名稱	資料形態	資料長度	索引
record_id	記錄流水號	int	-	PK
id	用戶流水號	int	-	FK
recognized_text	辨識句子	varchar	50	
recognized_at	辨識時間	datetime	-	

△表 8-2-4 recognize_record 之表格

第 9 章 程式

9-1 元件清單及其規格描述

表 9-1-1 前端程式清單

編號	檔案名稱	功能名稱
1	Index.html	首頁
2	About.html	關於我們
3	Learn.html	學習手語
4	Practice.html	即時手語翻譯
5	Profile.html	使用者資料
6	Video.html	播放影片
7	Login.html	登入
8	Logout.py	登出
9	register	註冊

表 9-1-2 後端程式清單

編號	檔案名稱	功能名稱
1	app.py	執行 HTML
2	create_db.py	建立資料庫
3	prepare_data_3d.py	資料前處理
4	extract_keypoints_3d.py	自動節點紀錄與標註

5	r_recognition_3d.py	即時辨識
6	train_transformer_3d.py	模型訓練
7	inference2.py	模型推論
8	Coordinate_ver.py	顯示節點數據
9	Video_download.py	影片下載
10	Finale.py	紀錄節點座標
11	VideoLabelTool.py	標記單詞和準確時間範圍
12	split_batch_keypoints.py	配對節點座標與 label
13	batch_trim_videos.py	批量影片剪輯

表 9-1-3 其他元件檔案

編號	檔案名稱	功能名稱
1	transformer_3d_best.h5	最佳模型版本
2	dataset_3d.npz	訓練與測試名單資料集
3	label_map.npy	label 索引對照表

9-2 重要程式碼

1. 影片轉節點：extract_keypoints_3d.py

extract_keypoints_3d.py	自動節點紀錄與標註
<pre>import os import cv2 import json import numpy as np import mediapipe as mp from tqdm import tqdm # === 基本設定 === VIDEO_DIR = "static/datavideos_2" # 影片來源資料夾 OUTPUT_DIR = "data/data_2" # 輸出 JSON 資料夾 os.makedirs(OUTPUT_DIR, exist_ok=True) MAX_FRAMES = 120 FEATURES = 21 * 3 * 2 + 68 * 3 # 330 維 (雙手+臉) # === 初始化 Mediapipe === mp_hands = mp.solutions.hands mp_face = mp.solutions.face_mesh hands = mp_hands.Hands(static_image_mode=False, max_num_hands=2)</pre>	

```

face_mesh = mp_face.FaceMesh(static_image_mode=False, max_num_faces=1)

# === 處理所有影片 ===

video_files = [f for f in os.listdir(VIDEO_DIR) if f.lower().endswith((".mp4", ".mov",
".avi"))]

print(f" 共發現 {len(video_files)} 支影片，開始處理...")

for video_name in video_files:

    label = os.path.splitext(video_name)[0]

    video_path = os.path.join(VIDEO_DIR, video_name)

    output_json = os.path.join(OUTPUT_DIR, f"{label}.json")

    cap = cv2.VideoCapture(video_path)

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    frames_data = []

    for fid in tqdm(range(total_frames), desc=f"處理 {video_name}", unit="frame"):

        ret, frame = cap.read()

        if not ret:

            break

        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        hand_results = hands.process(frame_rgb)

        face_results = face_mesh.process(frame_rgb)

```

```

# === 臉中心 ===

face_center = np.zeros(3)

face_landmarks = None

if face_results.multi_face_landmarks:

    face_landmarks = face_results.multi_face_landmarks[0]

    coords = np.array([[lm.x, lm.y, lm.z] for lm in
face_landmarks.landmark[:68]])

    face_center = coords.mean(axis=0)

# === 手部節點 ===

hand_points = []

if hand_results.multi_hand_landmarks:

    for hand_landmarks in hand_results.multi_hand_landmarks:

        coords = np.array([[lm.x, lm.y, lm.z] for lm in
hand_landmarks.landmark])

        coords = coords - face_center

        hand_points.extend(coords.flatten().tolist())

while len(hand_points) < 21 * 3 * 2:

    hand_points.extend([0.0] * (21 * 3))

# === 臉部節點 ===

face_points = []

if face_landmarks:

    coords = np.array([[lm.x, lm.y, lm.z] for lm in
face_landmarks.landmark[:68]])

```



```

        coords = coords - face_center

        face_points = coords.flatten().tolist()

    else:

        face_points = [0.0] * (68 * 3)

    # === 合併 ===

    frame_features = np.array(hand_points + face_points, dtype=np.float32)

    arr3 = frame_features.reshape(-1, 3)

    mean, std = arr3.mean(axis=0), arr3.std(axis=0)

    std[std == 0] = 1

    arr3 = (arr3 - mean) / std

    frames_data.append(arr3.flatten().tolist())

cap.release()

# === 補幀或裁切 ===

if len(frames_data) < MAX_FRAMES:

    pad = MAX_FRAMES - len(frames_data)

    frames_data.extend([[0.0] * FEATURES] * pad)

else:

    frames_data = frames_data[:MAX_FRAMES]

# === 儲存 JSON ===

with open(output_json, "w", encoding="utf-8") as f:

    json.dump({"label": label, "frames": frames_data}, f, ensure_ascii=False,
indent=2)

```

```
print(f' 已儲存：{output_json}')
```

```
print(" 所有影片已完成 3D 關鍵點擷取並以臉為中心平移！")
```

△表 9-2-1 extract_keypoints_3d.py

2. 資料前處理：prepare_data_3d.py

prepare_data_3d.py	資料前處理
<pre>import os import json import numpy as np from sklearn.model_selection import train_test_split # === 基本設定 === DATA_DIR = "data" # 存放所有 JSON 關鍵點資料的資料夾 OUTPUT_NPZ = os.path.join(DATA_DIR, "dataset_3d.npz") # 最終輸出的資料集檔案 OUTPUT_LABELS = os.path.join(DATA_DIR, "label_map.npy") # 對應的 label 映射表 MAX_FRAMES = 120 # 每支影片最多取多少幀 FEATURES = 21 * 3 * 2 + 68 * 3 # 每幀的特徵維度 (左右手 + 臉)</pre>	

```

# === 掃描所有 JSON 檔案 ===

files = [f for f in os.listdir(DATA_DIR) if f.endswith(".json")]

print(f' 發現 {len(files)} 個 JSON 檔案')


X, y = [], []

label_map = {}

idx = 0  # 用來給每個 label 編號


# === 處理每一個 JSON 檔 ===

for fname in files:

    fpath = os.path.join(DATA_DIR, fname)

    with open(fpath, "r", encoding="utf-8") as f:

        obj = json.load(f)


    # 原始標籤（影片對應的意思）


    # 有些 JSON 裡有 "label" 欄位，但我們更保險地根據檔名歸類

    label = obj.get("label", os.path.splitext(fname)[0])


    # 移除檔名中的數字，例如「早 1」、「早 2」→「早」

    label = ".join([c for c in label if not c.isdigit()]).strip()

```

```

# 確保 label 不為空

if not label:

    print(f' {fname} 沒有有效標籤，跳過")

    continue


frames = obj["frames"]

fixed_frames = []

# === 防呆修正：確保每幀長度一致 ===

for i, frame in enumerate(frames):

    flat = np.array(frame, dtype=np.float32).flatten()

    # 如果這一幀長度不符合預期，就補零或截斷

    if len(flat) != FEATURES:

        print(f' {fname} 第 {i} 幀長度異常 ({len(flat)}≠{FEATURES}) →
自動補零修正")

        if len(flat) < FEATURES:

            flat = np.pad(flat, (0, FEATURES - len(flat)), mode="constant")

        else:

            flat = flat[:FEATURES]

        fixed_frames.append(flat.tolist())

```

```

#=== 補幀或截斷：統一長度為 MAX_FRAMES ===

if len(fixed_frames) < MAX_FRAMES:

    pad = MAX_FRAMES - len(fixed_frames)

    fixed_frames.extend([[0.0] * FEATURES] * pad)

else:

    fixed_frames = fixed_frames[:MAX_FRAMES]

#=== 建立 label 對應表 ===

if label not in label_map:

    label_map[label] = idx

    idx += 1

X.append(fixed_frames)

y.append(label_map[label])

#=== 轉成 numpy 陣列 ===

X = np.array(X, dtype=np.float32)

y = np.array(y, dtype=np.int32)

#=== 建立 index 以便輸出檔名 ===

all_idx = np.arange(len(files))

#=== 分割訓練 / 測試資料 ===

X_train, X_test, y_train, y_test, X_train_idx, X_test_idx = train_test_split(

    X, y, all_idx,

```

```

    test_size=0.2,

    random_state=42,

    stratify=y if len(np.unique(y)) > 1 else None
)

# === 輸出 train/test 檔名清單 ===

train_files = [files[i] for i in X_train_idx]
test_files   = [files[i] for i in X_test_idx]


with open(os.path.join(DATA_DIR, "train_list.txt"), "w", encoding="utf-8") as f:

    for name in train_files:

        f.write(name + "\n")


with open(os.path.join(DATA_DIR, "test_list.txt"), "w", encoding="utf-8") as f:

    for name in test_files:

        f.write(name + "\n")


print(" Train/Test list 已輸出")


# === 儲存結果 ===

os.makedirs(DATA_DIR, exist_ok=True)

np.savez(OUTPUT_NPZ, X_train=X_train, X_test=X_test, y_train=y_train,
y_test=y_test)

np.save(OUTPUT_LABELS, label_map, allow_pickle=True)

```

```
# === 結果輸出 ===

print(f' Dataset 建立完成：{OUTPUT_NPZ}')

print(f' Label Map ( 共 {len(label_map)} 類 )：' )

print(label_map)
```

△表 9-2-2 prepare_data_3d.py

3. 訓練模型主程式：train_transformer_3d.py

train_transformer_3d.py	訓練模型主程式
<pre>import os import numpy as np import tensorflow as tf from tensorflow.keras import layers, models, optimizers, callbacks # === 檔案設定 === AUGMENTED_DATA = "data/dataset_3d_augmented.npz" ORIGINAL_DATA = "data/dataset_3d.npz" if os.path.exists(AUGMENTED_DATA): DATA_NPZ = AUGMENTED_DATA print(" 使用擴增後的 dataset_3d_augmented.npz") else: DATA_NPZ = ORIGINAL_DATA</pre>	

```

print(" 使用原始 dataset_3d.npz")

MODEL_OUT = "models/transformer_3d_best.h5"

# === 訓練參數 ===

EPOCHS = 80

BATCH_SIZE = 8

LR = 1e-4

# === 載入資料 ===

data = np.load(DATA_NPZ)

X_train, X_test = data["X_train"], data["X_test"]

y_train, y_test = data["y_train"], data["y_test"]

num_classes = len(np.unique(np.concatenate([y_train, y_test])))

seq_len, feat_dim = X_train.shape[1], X_train.shape[2]


print("\n===== 資料資訊 =====")

print(f"Train   X: {X_train.shape}")

print(f"Train   y: {y_train.shape}")

print(f"Test    X: {X_test.shape}")

print(f"Test    y: {y_test.shape}")

print(f"分類數量: {num_classes}")

print("===== \n")

```



```

# === Positional Encoding ===

class PositionalEncoding(layers.Layer):

    def __init__(self, length, dim):

        super().__init__()

        pos = np.arange(length)[: , None]

        i = np.arange(dim)[None, :]

        angle = pos / np.power(10000, (2 * (i // 2)) / dim)

        pe = np.zeros_like(angle)

        pe[:, 0::2] = np.sin(angle[:, 0::2])

        pe[:, 1::2] = np.cos(angle[:, 1::2])

        self.pe = tf.constant(pe[np.newaxis, ...], dtype=tf.float32)


    def call(self, x):

        return x + self.pe[:, :tf.shape(x)[1], :]


# === Transformer block ===

def transformer_block(x, heads=4, dim_ff=256, dropout=0.1):

    # Multi-head attention

    attn = layers.MultiHeadAttention(num_heads=heads, key_dim=64)(x, x)

    attn = layers.Dropout(dropout)(attn)

    x = layers.LayerNormalization()(x + attn)


    # Feed forward network

    ff = layers.Dense(dim_ff, activation="relu")(x)

```

```

ff = layers.Dense(x.shape[-1])(ff)

ff = layers.Dropout(dropout)(ff)

return layers.LayerNormalization()(x + ff)

# === 建立模型 ===

inp = layers.Input(shape=(seq_len, feat_dim))
x = PositionalEncoding(seq_len, feat_dim)(inp)

x = transformer_block(x, heads=4)
x = transformer_block(x, heads=4)

x = layers.GlobalAveragePooling1D()(x)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.3)(x)

out = layers.Dense(num_classes, activation="softmax")(x)

model = models.Model(inp, out)
model.compile(
    optimizer=optimizers.Adam(LR),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

```

```
print(" 模型架構：")

model.summary()

# === Callbacks ===

callbacks_list = [

    callbacks.EarlyStopping(

        patience=10,

        restore_best_weights=True,

        monitor="val_accuracy"

    ),

    callbacks.ModelCheckpoint(

        filepath=MODEL_OUT,

        save_best_only=True,

        monitor="val_accuracy",

        verbose=1,

        save_weights_only=False # 保存整個模型

    ),

    callbacks.ReduceLROnPlateau(

        monitor="val_loss",

        factor=0.5,

        patience=5,

        min_lr=1e-6,

        verbose=1
```

```

    )
]
os.makedirs("models", exist_ok=True)

# === 訓練 ===

print("\n 開始訓練模型...\n")

history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=callbacks_list,
    verbose=1
)

print(f"\n 訓練完成！最好的模型已儲存至：{MODEL_OUT}")

```

△圖 9-2-3 train_transformer_3d.py

4. 即時辨識：r_recognition_3d.py

r_recognition_3d.py	即時辨識
<pre> import os import cv2 import numpy as np </pre>	

```

import time

import mediapipe as mp

import tensorflow as tf

from tensorflow.keras.models import load_model

from tensorflow.keras import layers

from PIL import ImageFont, ImageDraw, Image

# === 模型與資料路徑 ===

MODEL_PATH = "models/transformer_3d_best.h5"

LABEL_MAP_PATH = "data/label_map.npy"


MAX_FRAMES = 120

FEATURES = 21 * 3 * 2 + 68 * 3  # 雙手 + 臉部 68 點


# === 字型設定 (Windows 範例: 微軟正黑體) ===

font_path = "C:/Windows/Fonts/msjh.ttc"

font = ImageFont.truetype(font_path, 32)


# === 自訂層：Positional Encoding (必須加入以便 load_model) ===

class PositionalEncoding(layers.Layer):

    def __init__(self, length, dim, **kwargs):

        super(PositionalEncoding, self).__init__(**kwargs)

        pos = np.arange(length)[:, None]

        i = np.arange(dim)[None, :]

        angle = pos / np.power(10000, (2 * (i // 2)) / dim)

```

```

        pe = np.zeros_like(angle)

        pe[:, 0::2] = np.sin(angle[:, 0::2])

        pe[:, 1::2] = np.cos(angle[:, 1::2])

        self.pe = tf.constant(pe[np.newaxis, ...], dtype=tf.float32)

    def call(self, x):

        return x + self.pe[:, :tf.shape(x)[1], :]

# === 檢查模型是否存在 ===

if not os.path.exists(MODEL_PATH):

    raise FileNotFoundError(

        f" 找不到模型 {MODEL_PATH}\n 請確認 train_transformer_3d.py 已

        成功訓練並輸出模型。"

    )

print(f" 載入模型：{MODEL_PATH}")

# === 載入模型 ===

with tf.keras.utils.custom_object_scope({

    'PositionalEncoding': PositionalEncoding

}):

    model = load_model(MODEL_PATH)

# === 載入標籤映射 ===

label_map = np.load(LABEL_MAP_PATH, allow_pickle=True).item()

```

```

inv_map = {v: k for k, v in label_map.items()}

# === 初始化 Mediapipe ===

mp_hands = mp.solutions.hands
mp_face = mp.solutions.face_mesh
hands = mp_hands.Hands(max_num_hands=2)
face_mesh = mp_face.FaceMesh(max_num_faces=1)

# === 開啟攝影機 ===

cap = cv2.VideoCapture(0)
buff = []

print(" 開始即時辨識（按 Q 離開）")

# === 顯示控制與穩定機制 ===

last_label = "搜尋中"

last_confidence = 0.0
last_success_time = 0

display_duration = 2.5  # 成功結果顯示秒數

CONF_THRESHOLD = 0.8    # 信心門檻

success_count = 0

REQUIRED_SUCCESS_FRAMES = 5  # 需連續幾幀才算成功

while True:

```

```

ret, frame = cap.read()

if not ret:
    break

rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
hand_results = hands.process(rgb)
face_results = face_mesh.process(rgb)

# --- 計算臉部中心 ---

face_center = np.zeros(3)
face_landmarks = None

if face_results.multi_face_landmarks:
    face_landmarks = face_results.multi_face_landmarks[0]

    coords = np.array([[lm.x, lm.y, lm.z] for lm in
face_landmarks.landmark[:68]])

    face_center = coords.mean(axis=0)

# --- 取得雙手特徵 ---

hand_points = np.zeros(21*3*2, dtype=np.float32)

if hand_results.multi_hand_landmarks:
    for idx, hand_landmarks in enumerate(hand_results.multi_hand_landmarks):
        if idx >= 2:
            break

        coords = np.array([[lm.x, lm.y, lm.z] for lm in
hand_landmarks.landmark]).flatten()

        hand_points[idx*21*3:(idx+1)*21*3] = coords - np.tile(face_center,

```


21)

```
# --- 取得臉部特徵 ---

face_points = np.zeros(68*3, dtype=np.float32)

if face_landmarks:

    coords = np.array([[lm.x, lm.y, lm.z] for lm in
face_landmarks.landmark[:68]]).flatten()

    face_points = coords - np.tile(face_center, 68)

# --- 特徵組合 + 標準化 ---

frame_features = np.concatenate([hand_points, face_points])

arr3 = frame_features.reshape(-1, 3)

mean, std = arr3.mean(axis=0), arr3.std(axis=0)

std[std == 0] = 1

arr3 = (arr3 - mean) / std

frame_features = arr3.flatten()

buff.append(frame_features)

if len(buff) > MAX_FRAMES:

    buff.pop(0)

# --- 推論與顯示 ---

pil_img = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

draw = ImageDraw.Draw(pil_img)
```

```

current_time = time.time()    # 一開始就取得正確時間

#=== 檢查是否有手 ===

if not hand_results.multi_hand_landmarks:

    # 沒有手：直接顯示搜尋中

    display_text = "搜尋中"

    display_conf = None


    # 清空 buffer，避免殘留舊手勢造成誤判

    buff = []


    # 畫文字後跳過此迴圈 → 完全不做模型推論

    draw.text((30, 30), display_text, font=font, fill=(255, 255, 0))

    frame = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)

    cv2.imshow("Sign Recognition", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):

        break

    continue

# --- 若 buffer 已滿，進行推論 ---

if len(buff) >= MAX_FRAMES:

```

```

seq = buff[-MAX_FRAMES:]

X = np.expand_dims(np.array(seq, dtype=np.float32), 0)

pred = model.predict(X, verbose=0)

confidence = float(np.max(pred))

label_candidate = inv_map[int(np.argmax(pred))]

# --- 高信心：視為成功辨識 ---

if confidence > CONF_THRESHOLD:

    # 記錄成功辨識

    last_label = label_candidate

    last_confidence = confidence

    last_success_time = current_time    # 使用正確時間

# --- 顯示邏輯 ---

time_since_last = current_time - last_success_time

if time_since_last > display_duration:

    display_text = "搜尋中"

    display_conf = None

else:

    display_text = last_label

```

```

display_conf = last_confidence

buff = []    # 重新開始收集新手勢

# --- 畫文字 ---

if display_conf is not None:

    draw.text((30, 30), f'{display_text} ({display_conf*100:.1f}%)',

              font=font, fill=(0, 255, 0))

else:

    draw.text((30, 30), display_text,

              font=font, fill=(255, 255, 0))


frame = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)

# --- 畫出手部 / 臉部 ---

if hand_results.multi_hand_landmarks:

    for hand_landmarks in hand_results.multi_hand_landmarks:

        mp.solutions.drawing_utils.draw_landmarks(

            frame, hand_landmarks, mp_hands.HAND_CONNECTIONS,

            mp.solutions.drawing_utils.DrawingSpec(color=(0,0,0),

thickness=2, circle_radius=2),

            mp.solutions.drawing_utils.DrawingSpec(color=(0,0,0),

thickness=2)

        )

```

```

# --- 畫出臉部網格 (黑色) ---

if face_landmarks:

    mp.solutions.drawing_utils.draw_landmarks(

        frame, face_landmarks, mp_face.FACEMESH_TESSELATION,

        mp.solutions.drawing_utils.DrawingSpec(color=(0,0,0), thickness=1,
circle_radius=1),

        mp.solutions.drawing_utils.DrawingSpec(color=(0,0,0), thickness=1)

    )

# --- 顯示畫面 ---

cv2.imshow("Sign Recognition", frame)

if cv2.waitKey(1) & 0xFF == ord("q"):

    break

cap.release()

cv2.destroyAllWindows()

```

△表 9-2-4 r_recognition_3d.py

第十章 測試模型

10-1 測試計畫：說明採用之測試方法及其進行方式

- 測試平台：使用 Sign-Nest 手語小窩進行多平台及多瀏覽器的相容性測試
- 測試方式：針對系統的模組進行測試，模擬使用者操作並檢查系統是否可順利運行
- 測試目標：驗證系統的穩定性，準確性及使用者體驗，確保系統正常操作且符合要求

1. 帳號

(1) 註冊：測試是否能夠註冊新帳號。

(2) 登入：測試是否能夠登入。

2. 手語教學功能：

(1) 測試影片是否能正常播放。

3. 手語辨識功能：

(1) 測試是否可辨識手語所代表的意思。

● 測試方法：

1. 將每個功能進行測試，確保系統穩定性

2. 測試過程中的流程與結果將會被記錄，以確保每個功能都符合需求

- 開發環境：HTML, CSS (前端), Python, Flask (後端)
- 瀏覽器：測試將涵蓋主要瀏覽器。
- 設備：桌面電腦，筆記型電腦。

10-2 測試個案與測試結果資料

表 10-2-1 會員註冊功能測試

功能名稱	會員註冊
測試目標	確認用戶能夠註冊
測試流程	1. 輸入用戶名，信箱及密碼 2. 驗證資料庫是否有重複的會員資料 3. 如無重複的會員資料，則將該會員資料註冊
測試結果	成功

表 10-2-2 會員登入功能測試

功能名稱	會員登入
測試目標	確認用戶能夠登入帳號
測試流程	1. 輸入用戶名，信箱及密碼 2. 驗證資料庫是否有該會員資料 3. 如無該會員資料，則需先註冊帳號
測試結果	成功

表 10-2-3 密碼重設功能測試

功能名稱	密碼重設
測試目標	確認用戶忘記密碼時可重設密碼
測試流程	1. 點擊忘記密碼 2. 輸入用戶名及信箱 3. 確認是否收到驗證碼 4. 是否能成功使用新密碼登入
測試結果	成功

表 10-2-4 影片播放功能測試

功能名稱	影片播放
測試目標	確認手語課程中的影片是否能夠播放
測試流程	1. 輸入用戶名，信箱及密碼並登入帳號 2. 點選課程功能 3. 點擊影片並播放
測試結果	成功

表 10-2-5 詞彙分類功能測試

功能名稱	詞彙分類
測試目標	可從詞彙分類欄查找到手語影片
測試流程	1. 輸入用戶名，信箱及密碼 2. 登入帳號 3. 點選其中一個詞彙分類 4. 顯示該分類中的手語詞彙
測試結果	成功

表 10-2-6 上傳教學影片功能測試

功能名稱	上傳教學影片
測試目標	可從上傳影片類別上傳影片
測試流程	1. 輸入管理員用戶名，信箱及密碼 2. 登入帳號 3. 點選上傳影片
測試結果	成功

表 10-2-7 手語辨識功能測試

功能名稱	手語辨識
測試目標	可辨識用戶所比的手語並輸出對應詞匯
測試流程	1. 輸入用戶名，信箱及密碼並登入帳號 2. 點選即時翻譯並打開攝像頭 3. 用戶比出手語並輸出對應詞匯
測試結果	成功

表 10-2-8 查詢詞語功能測試

功能名稱	查詢詞語
測試目標	可查詢該詞語所對應的手語教學影片
測試流程	1. 輸入用戶名，信箱及密碼 2. 登入帳號 3. 點選搜尋欄並輸入詞語 4. 顯示詞語所對應的手語教學影片
測試結果	成功

第 11 章 操作手冊

11-1 系統元件

表 11-1-1 系統安裝元件資訊

系統名稱	SignNest-手語小窩
版本	V1.0
支援語言	繁體中文
價格	免費
最低版本需求	聯網裝置
內容分級	普遍級

11-2 系統需求

- 網頁瀏覽器:Chrome, Firefox, Safari 或 Brave Browser
- 穩定的網際網路連線

第 12 章 使用手冊

表 12-1-1 登入介面

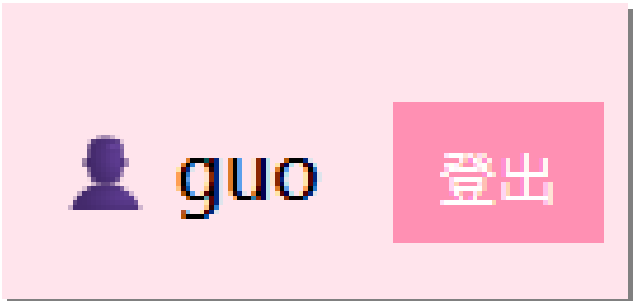
進入 SignNest-手語小窩的登入介面	
 The login interface features a light blue header with the title '登入' (Login) in red. Below the header, there are two input fields: '電子郵件:' (Email) and '密碼:' (Password). A red '登入' (Login) button is positioned below the password field. At the bottom, a link '還沒有帳號? 註冊' (Don't have an account? Register) is displayed in red.	 This section shows a user profile with a blue silhouette icon and the name 'guo' in a stylized font. To the right, there is a red button with the white text '登出' (Logout).
選擇登入	成功登入

表 12-1-2 註冊介面

進入 SignNest-手語小窩的註冊介面	
	 The registration interface has a light blue header with the title '註冊' (Register) in red. The form includes several fields: '電子郵件:' (Email), '姓名:' (Name), '性別:' (Gender) with a dropdown menu showing '男' (Male), '年齡:' (Age), '密碼:' (Password), and '確認密碼:' (Confirm Password). A red '註冊' (Register) button is at the bottom. A link '已經有帳號了? 請登入' (Already have an account? Please login) is located at the very bottom in red.
註冊資訊介面	

表 12-1-3 首頁

進入 SignNest-手語小窩的首頁



表 12-1-4 關於我們介面

進入 SignNest-手語小窩的關於我們介面

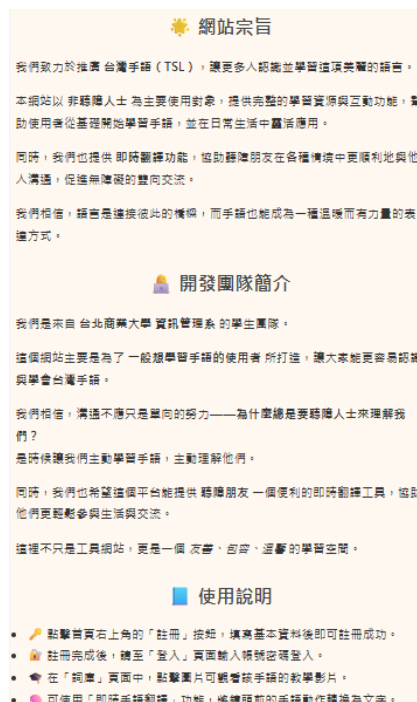


表 12-1-5 學習手語介面

進入 SignNest-手語小窩的學習手語介面	
	
選擇學習的詞彙	播放手語教學影片

表 12-1-6 即時翻譯手語介面

進入 SignNest-手語小窩的即時翻譯手語介面		
		
打開鏡頭	比出需翻譯的手語	顯示出辨識結果

表 12-1-7 即時翻譯範例

翻譯畫面 搜尋中 執行中 開始 結束	翻譯畫面 沒關係 (86.3%) 執行中 開始 結束	翻譯結果 目前辨識的單字 沒關係 說明 - 按「開始」後，系統會開啟攝影機並啟動後端 Transformer 模型。 - 框內畫面已包含：手部關節、臉部網格與模型預測文字。 - 右側會顯示目前辨識到的手語單字。 小提醒：若畫面沒有更新，請確認攝影機權限是否允許，或重新整理頁面再試一次。
翻譯畫面 搜尋中 執行中 開始 結束	翻譯畫面 不見 (94.9%) 執行中 開始 結束	翻譯結果 目前辨識的單字 不見 說明 - 按「開始」後，系統會開啟攝影機並啟動後端 Transformer 模型。 - 框內畫面已包含：手部關節、臉部網格與模型預測文字。 - 右側會顯示目前辨識到的手語單字。 小提醒：若畫面沒有更新，請確認攝影機權限是否允許，或重新整理頁面再試一次。
翻譯畫面 搜尋中 執行中 開始 結束	翻譯畫面 錢包 (99.2%) 執行中 開始 結束	翻譯結果 目前辨識的單字 錢包 說明 - 按「開始」後，系統會開啟攝影機並啟動後端 Transformer 模型。 - 框內畫面已包含：手部關節、臉部網格與模型預測文字。 - 右側會顯示目前辨識到的手語單字。 小提醒：若畫面沒有更新，請確認攝影機權限是否允許，或重新整理頁面再試一次。
翻譯畫面 搜尋中 執行中 開始 結束	翻譯畫面 傳輸 (83.5%) 執行中 開始 結束	翻譯結果 目前辨識的單字 傳輸 說明 - 按「開始」後，系統會開啟攝影機並啟動後端 Transformer 模型。 - 框內畫面已包含：手部關節、臉部網格與模型預測文字。 - 右側會顯示目前辨識到的手語單字。 小提醒：若畫面沒有更新，請確認攝影機權限是否允許，或重新整理頁面再試一次。

表 12-1-8 使用者資訊介面

進入 SignNest-手語小窩的使用者資訊介面	
 個人資料 姓名：guo 性別：女 年齡：22 Email：11146038@ntub.edu.tw	 您已成功登出 期待再次見到您！ 重新登入 回首頁
使用者資訊介面	使用者登出
 編輯個人資料 姓名 guo 性別 女 年齡 22 新密碼 不修改請留空 確認新密碼 不修改請留空 儲存修改	
更改使用者資訊	

第 13 章 感想

11046042 周駿翔:

這次專題我擔任組長，題目是手語辨識，原因是剛好我們有組員是聽障生，因此我們便很快地決定做相關主題的專題。雖然身為組長，但我們組員其實彼此較不熟悉，大家都很害羞安靜，而我本身個性也較內向，鮮少擔任指揮的角色，這對我來說是一大挑戰，起初也因我分配工作不明確導致工作比重失衡，這讓我理解到身為管理者的困難與挑戰。

原本老師擔心辨識功能較困難，擔心我們會開天窗，所以建議加入教學網站的概念，但經評審建議，認為範圍太大，進而決定專心做辨識的部分，從一開始的資料蒐集、拍影片，何謂可以用的資料?該如何做資料標記?如何做資料處理?如何訓練?該用什麼模型?雖然課堂都有大略上過，但一路上實際做出來才知道一切都是未知，所有的環節都沒有正確答案，都要一一測試才知道何謂是好的結果，很感謝俊杰老師很耐心地為我們講解，也很不辭辛勞地陪我們開會。雖然辨識結果不如我們預期，但經過這次專題，我更了解整個系統的從無到有是如何製作的，也真正明白到團隊的重要性。

11146038 郭子寧:

我覺得做專題真的很不容易，從零到有的過程充滿挑戰。雖然我們不是成績最好的一組，但至少我們盡了全力，這讓我感到非常有成就感。

透過兩學期的專題時光，我學到了很多課堂上學不到的知識和技能。從一開始的分組、討論主題、規劃製作流程，再到分工合作，每個環節都充滿了學習的機會。

我們嘗試架設網站、設計前端介面、連接後端功能，甚至進行 AI 模型的訓練，這些過程讓我對技術的理解更加深刻。雖然過程中遇到不少困難，例如程式錯誤、系統整合的挑戰，但每當看到組員和自己一點一點完成任務、克服困難，就會有很大的成就感。

除了技術能力的提升，我也學到如何與組員有效合作、如何分工協調，這些都是未來職場中非常重要的技能。專題讓我明白，學習的過程本身比結果更有價值，只要盡力去做，即使遇到錯誤，也是一種寶貴的經驗。

總結來說，這次專題經驗不僅提升了我的專業能力，也讓我對團隊合作、問題解決和持續學習有了更深的體會。雖然挑戰不少，但這段經歷絕對是我大學生活中非常珍貴的回憶。

11146003 黃俊翰:

在專題初審前，我一直專心地開發程式，沒想到在過程中遇到了許多環境設定與程式上的障礙。原本我想自己解決，但後來發現有些問題需要與組員合作，才能更快速有效地完成。每位組員都有不同的特點與專長，透過討論與分工，我們成功克服了許多困難。這次專題對我有非常深的意義。因為我自己是聽障者，我最能體會溝通上的不便，因此特別希望這個系統能真正幫助到和我有相同需求的族群。從收集手語影片、訓練模型到實作即時翻譯，每一步都充滿挑戰，整個程式的開發過程其實並不容易，但我在其中學到了很多，也感受到努力後的成果。能把自己的親身經驗轉化成實際的技術成果，幫助他人理解手語，對我來說是最大的成就。我也希望未來能持續推廣這個系統，讓更多人能夠方便地使用手語溝通，讓世界變得更包容。

11146040 黃政:

在構思專題題目的時候，我們一開始其實還不確定要做什麼樣的主題。不過，因為我們其中一位組員是聽障生，與他溝通時需要透過打字進行交流，而我們又看不懂、也不會使用手語，因此便萌生了「製作手語辨識系統」這個想法。

在專題初期，我主要負責資料庫的建立與部分文件撰寫。經過初評後，我開始接手更多後端相關的工作，尤其是在負責手部與臉部節點辨識的階段時，常常為了debug 而熬夜。即使在打工回家後的時間，我也都投入在程式撰寫上。當系統成果還未達預期或結果不理想時，我內心也不斷掙扎與焦慮，擔心專題無法順利完成。不過，也正是在這些一次又一次的後端開發與調整過程中，我的程式能力逐漸提升，雖然稱不上多厲害，但至少能夠獨立完成任務。

在這份專題中，我的主要工作是負責資料庫設計與協助撰寫後端程式。非常感謝我們的指導老師，在我們迷茫、不知道該往哪個方向發展的時候，給予了明確的建議與範例指導，例如提醒我們臉部辨識可以採用「基於臉部的相對座標」會更為準確。最後，也要感謝組員們在這段期間的努力與配合，大家都為這份專題投入了十二分的心力。

第 14 章 參考資料

1. pytorch:
<https://pytorch.org/>
2. 什麼是人工智慧的 Transformer:
<https://aws.amazon.com/tw/what-is/transformers-in-artificial-intelligence/>
3. Text generation via RNN and LSTMs (PyTorch):
<https://www.kaggle.com/code/purvasingh/text-generation-via-rnn-and-lstms-pytorch>
4. pytorch-lstm-text-generation-tutorial:
<https://github.com/closeheat/pytorch-lstm-text-generation-tutorial/tree/master>
5. 丁立芬
<https://www.youtube.com/@tingangle/playlists>
6. 遞歸神經網路 (RNN) 和長短期記憶模型 (LSTM) 的運作原理
https://brohrer.mcknote.com/zh-Hant/how_machine_learning_works/how_rnn_lstm_work.html
7. Self-attention(自注意力機制)
<https://medium.com/@x02018991/%E6%A9%9F%E5%99%A8%E5%AD%B8%E7%BF%92%E7%AD%86%E8%A8%98-self-attention-fa6897080a0a>

附錄

審查評審意見之修正情形（初評及複評之評審建議項，均請填寫）

評審建議事項		修正情形
初評	題目範圍太大，教學、辨識鎖定一個就好，且即時翻譯兼教學，這樣應該是翻譯的功能大於教學，但目前教學大約翻譯	已暫停教學模組功能，僅保留觀看手語教學影片功能，著重翻譯訓練
	只有俊翰單一的樣本數太少，是否要提高母數來提升翻譯的準確度，正確性、召回率、精確度，是否有要求這一塊	除了俊翰外，全組組員均有拍攝自身的手語影片，也藉此提供不同樣本供模型訓練，藉此提高正確率
複評	目前完成度不高，是否縮小範圍，配合教學改成以測驗方式呈現相似度，開啟鏡頭比手語，再告訴使用者與正確姿勢相比有多少相似度	與指導老師討論後，因與當初題目設定方向不符，相似度是另外一個領域與方向，我們是期望做連續即時翻譯，所以所製作的模型、程式都與相似度不同，相似度是不同的模型，時程來不及，故依舊以即時翻譯為主
	模型使用的是一維，是否使用二維或三維模型來訓練，例如 Yolo，來提高正確率	我們的訓練資料是三維的，會以一維模型做後續訓練是因與指導老師討論，認為不需要使用到

		二、三維模型，結果正確率有達到 97%，後續嘗試過增加節點的移動向量，結果正確率降為 80%，故依舊維持一維的模型
--	--	--